Авторизация и регистрация. Превварительные процессы

Основные подходы:

- оркестрация процесса со стороны бэкенда
- для каждого устройства выпускается своя пара ключей device private/public key - device key. Ключи используются как подтверждение устройства (подисываются запросы). Public key отправляется и хранится на сервере
- device id (уникальный идентификатор устройства) - генерируется на фронте (спец.библиотеками или функциями ОС), к нему привязан уникальный device public key
- PIN не хранится на сервере, это не пароль пользователя, а локальный код для разблокировки самого устройства (аналогия - FaceId, TouchId)
- при использовании устройства разными пользователями: хранятся метаданные только последнего пользователя, данные других затираются

Полная авторизация. Предварительный процесс

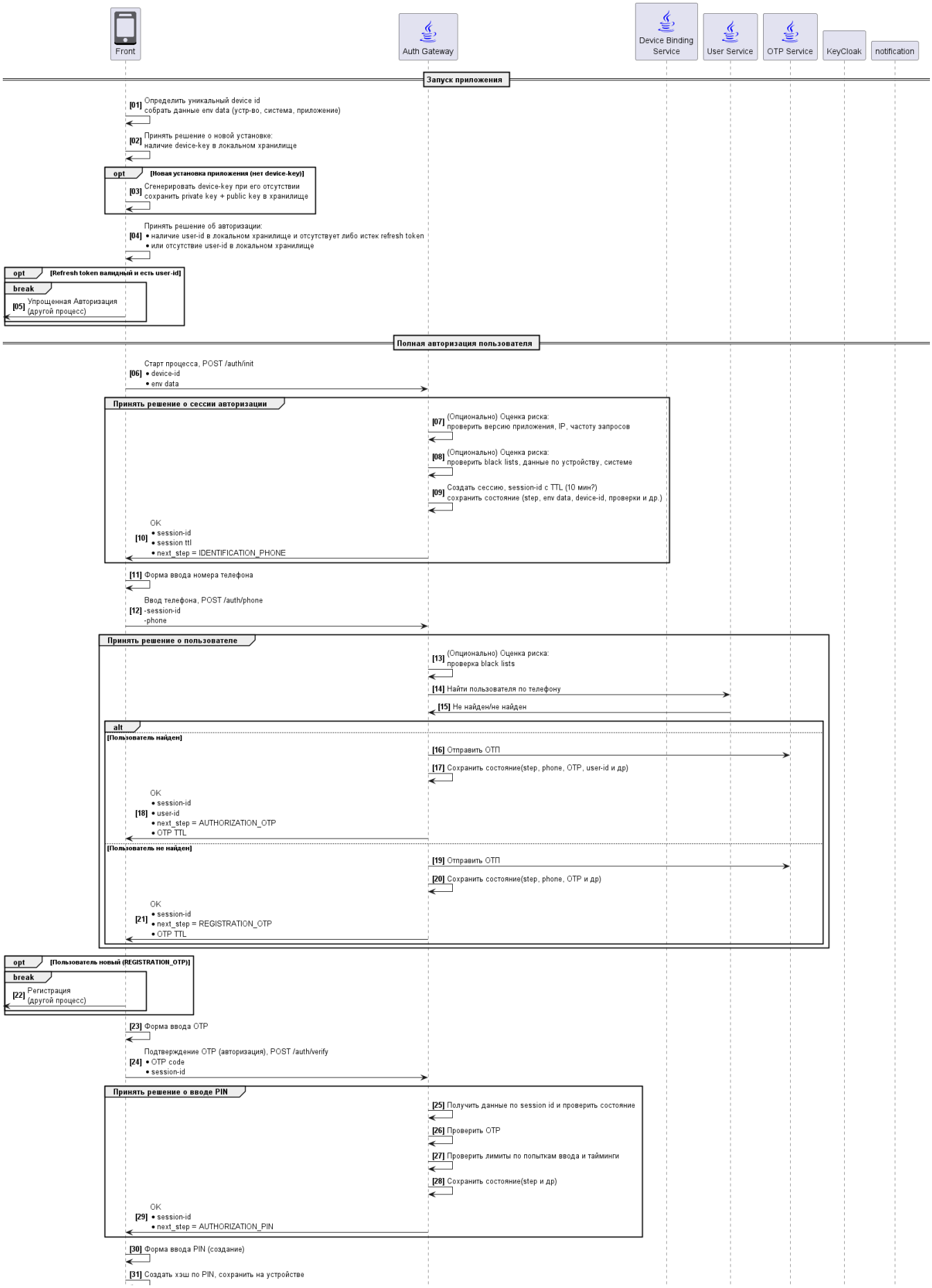
Сценарии:

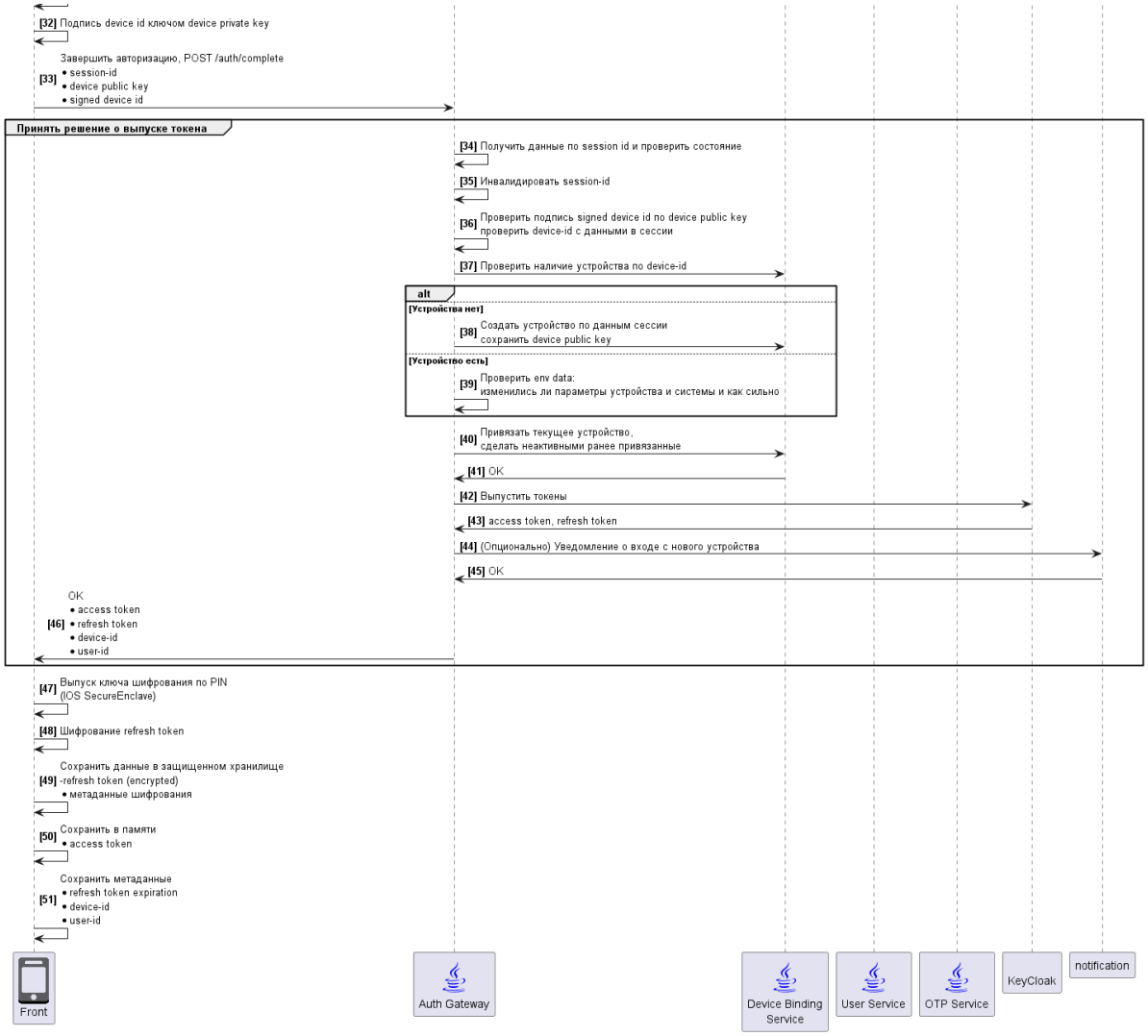
- первая установка приложения, пользователь зарегистрирован
- авторизация второго пользователя в установленном приложении

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<font color=blue><$java>\nAuth Gateway" as auth
participant "<font color=blue><$java>\nDevice Binding\nService" as device
participant "<font color=blue><$java>\nUser Service" as user
participant "<font color=blue><$java>\nOTP Service" as otp
participant "<font color=blue>\nKeyCloak" as idp

autonumber "<b>[00]"
==   ==

  frontend -> frontend:   device id\n  env data (-, , )

  frontend -> frontend:     :\n device-key

  opt    ( device-key)
    frontend -> frontend:  device-key   \n private key + public key
  end opt

  frontend -> frontend:    :\n* user-id        refresh token\n*  user-id

  opt Refresh token    user-id
    break
      frontend ->[: \n( )
    end break
  end opt

==    ==

  frontend -> auth:  , POST /auth/init\n*device-id\n*env data
  group
    auth -> auth: ()  : \n  , IP,
    auth -> auth: ()  : \n black lists,   ,

    auth -> auth:  , session-id  TTL (10 ?)\n  (step, env data, device-id,   .)

    auth -> frontend: OK\n*session-id\n*session ttl\n*next_step = IDENTIFICATION_PHONE
  end

  frontend -> frontend:
  frontend -> auth:  , POST /auth/phone\n-session-id\n-phone
  group

    auth -> auth: ()  :\n black lists

    auth -> user:
    user -> auth:  /
    alt
      else
        auth -> otp:
```

```
        auth -> auth:  (step, phone, OTP, user-id  )
        auth -> frontend: OK\n*session-id\n*user-id\n*next_step = AUTHORIZATION_OTP\n*OTP TTL
      else
        auth -> otp:
        auth -> auth:  (step, phone, OTP  )
        auth -> frontend: OK\n*session-id\n*next_step = REGISTRATION_OTP\n*OTP TTL
    end
  end


  opt   (REGISTRATION_OTP)
    break
      frontend ->[:\n( )
    end break
  end opt

    frontend -> frontend:   OTP
    frontend -> auth:  OTP (), POST /auth/verify\n*OTP code\n*session-id
    group     PIN
      auth -> auth:    session id
      auth -> auth:  OTP

      auth -> auth:

      auth -> auth:  (step  )

      auth -> frontend: OK\n*session-id\n*next_step = AUTHORIZATION_PIN

    end

    frontend -> frontend:   PIN ()
    frontend -> frontend:    PIN,
    frontend -> frontend:  device id  device private key

    frontend -> auth:  , POST /auth/complete\n*session-id\n*device public key\n*signed device id
    group
      auth -> auth:    session id

      auth -> auth:  session-id

      auth -> auth:   signed device id  device public key\n device-id

      auth -> device:     device-id
      alt
        else
          auth -> device:     \n device public key
        else
          auth -> auth:  env data:\n
      end

      auth -> device:   ,\n
      device -> auth: OK

      auth -> idp:
      idp -> auth: access token, refresh token

      auth -> notification: ()
      notification -> auth: OK

      auth -> frontend: OK\n*access token\n*refresh token\n*device-id\n*user-id

    end

    frontend -> frontend:     PIN\n(IOS SecureEnclave)
    frontend -> frontend:  refresh token
    frontend -> frontend:     \n-refresh token (encrypted)\n*
    frontend -> frontend:   \n*access token
    frontend -> frontend:  \n*refresh token expiration\n*device-id\n*user-id

@enduml
```


Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
notification
Запуск приложения
[01] Определить уникальный device id
собрать данные env data (устр-во, система, приложение)
[02] Принять решение о новой установке:
наличие device-key в локальном хранилище
opt [Новая установка приложения (нет device-key)]
[03] Сгенерировать device-key при его отсутствии
сохранить private key + public key в хранилище
Принять решение об авторизации:
[04] • наличие user-id в локальном хранилище и отсутствует либо истек refresh token
• или отсутствие user-id в локальном хранилище
opt [Refresh token валидный и есть user-id]
break
[05] Упрощенная Авторизация
(другой процесс)
Полная авторизация пользователя
Старт процесса, POST /auth/init
[06] • device-id
• env data
Принять решение о сессии авторизации
[07] (Опционально) Оценка риска:
проверить версию приложения, IP, частоту запросов
[08] (Опционально) Оценка риска:
проверить black lists, данные по устройству, системе
[09] Создать сессию, session-id с TTL (10 мин?)
сохранить состояние (step, env data, device-id, проверки и др.)
OK
[10] • session-id
• session ttl
• next_step = IDENTIFICATION_PHONE
[11] Форма ввода номера телефона
Ввод телефона, POST /auth/phone
[12] -session-id
-phone
Принять решение о пользователе
[13] (Опционально) Оценка риска:
проверка black lists
[14] Найти пользователя по телефону
[15] Не найден/не найден
alt
[Пользователь найден]
[16] Отправить ОТП
[17] Сохранить состояние(step, phone, OTP, user-id и др)
OK
[18] • session-id
• user-id
• next_step = AUTHORIZATION_OTP
• OTP TTL
[Пользователь не найден]
[19] Отправить ОТП
[20] Сохранить состояние(step, phone, OTP и др)
OK
[21] • session-id
• next_step = REGISTRATION_OTP
• OTP TTL
opt [Пользователь новый (REGISTRATION_OTP)]
break
[22] Регистрация
(другой процесс)
[23] Форма ввода OTP
Подтверждение OTP (авторизация), POST /auth/verify
[24] • OTP code
• session-id
Принять решение о вводе PIN
[25] Получить данные по session id и проверить состояние
[26] Проверить OTP
[27] Проверить лимиты по попыткам ввода и тайминги
[28] Сохранить состояние(step и др)
OK
[29] • session-id
• next_step = AUTHORIZATION_PIN
[30] Форма ввода PIN (создание)
[31] Создать хэш по PIN, сохранить на устройстве


[32] Подпись device id ключом device private key
Завершить авторизацию, POST /auth/complete
[33]
• session-id
• device public key
• signed device id
Принять решение о выпуске токена
[34] Получить данные по session id и проверить состояние
[35] Инвалидировать session-id
[36] Проверить подпись signed device id по device public key
проверить device-id с данными в сессии
[37] Проверить наличие устройства по device-id
alt
[Устройства нет]
[38] Создать устройство по данным сессии
сохранить device public key
[Устройство есть]
[39] Проверить env data:
изменились ли параметры устройства и системы и как сильно
[40] Привязать текущее устройство,
сделать неактивными ранее привязанные
[41] OK
[42] Выпустить токены
[43] access token, refresh token
[44] (Опционально) Уведомление о входе с нового устройства
[45] OK
OK
[46]
• access token
• refresh token
• device-id
• user-id
[47] Выпуск ключа шифрования по PIN
(IOS SecureEnclave)
[48] Шифрование refresh token
Сохранить данные в защищенном хранилище
[49]
-refresh token (encrypted)
• метаданные шифрования
Сохранить в памяти
[50]
• access token
Сохранить метаданные
[51]
• refresh token expiration
• device-id
• user-id
Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
notification

Полная авторизация. Предварительный процесс

Сценарии:

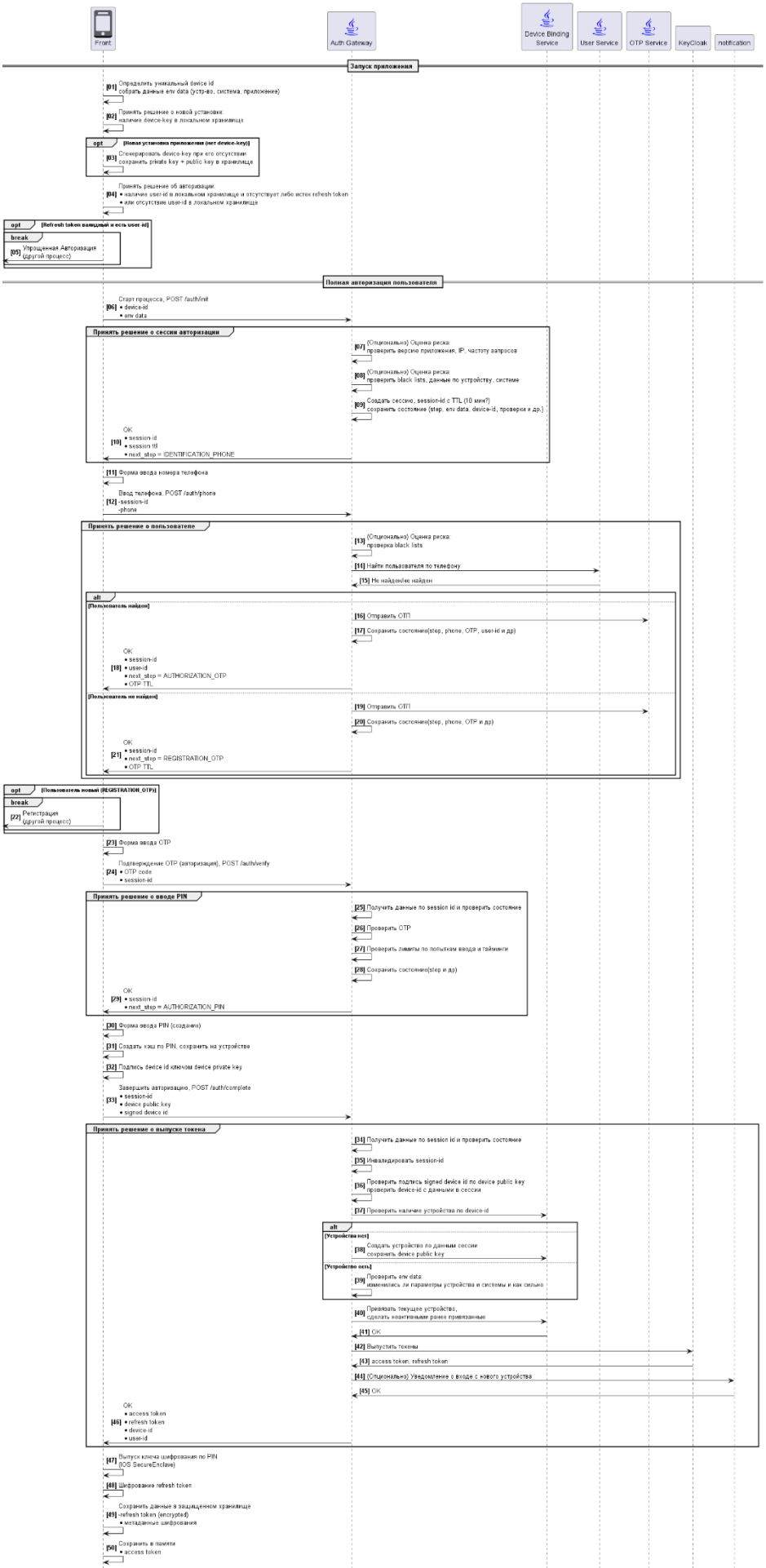
- первая установка приложения, пользователь зарегистрирован
- авторизация второго пользователя в установленном приложении

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<font color=blue><$java>\nAuth Gateway" as auth
participant "<font color=blue><$java>\nDevice Binding\nService" as device
participant "<font color=blue><$java>\nUser Service" as user
participant "<font color=blue><$java>\nOTP Service" as otp
participant "<font color=blue>\nKeyCloak" as idp

autonumber "<b>[00]"
== Запуск приложения ==

  frontend -> frontend: Определить уникальный device id\nсобрать данные env
data (устр-во, система, приложение)

  frontend -> frontend: Принять решение о новой установке:\nналичие device-
key в локальном хранилище

  opt Новая установка приложения (нет device-key)
    frontend -> frontend: Сгенерировать device-key при его
отсутствии\nсохранить private key + public key в хранилище
  end opt

  frontend -> frontend: Принять решение об авторизации:\n*наличие user-id в
локальном хранилище и отсутствует либо истек refresh token\n*или отсутствие
user-id в локальном хранилище

  opt Refresh token валидный и есть user-id
    break
      frontend ->[:Упрощенная Авторизация\n(другой процесс)
    end break
  end opt
```

```
== Полная авторизация пользователя ==

  frontend -> auth: Старт процесса, POST /auth/init\n*device-id\n*env data
  group Принять решение о сессии авторизации
    auth -> auth: (Опционально) Оценка риска: \nпроверить версию приложения,
IP, частоту запросов
    auth -> auth: (Опционально) Оценка риска: \nпроверить black lists, данные
по устройству, системе

    auth -> auth: Создать сессию, session-id с TTL (10 мин?)\nсохранить
состояние (step, env data, device-id, проверки и др.)

    auth -> frontend: OK\n*session-id\n*session ttl\n*next_step =
IDENTIFICATION_PHONE
  end

  frontend -> frontend: Форма ввода номера телефона
  frontend -> auth: Ввод телефона, POST /auth/phone\n-session-id\n-phone
  group Принять решение о пользователе

    auth -> auth: (Опционально) Оценка риска:\nпроверка black lists

    auth -> user: Найти пользователя по телефону
    user -> auth: Не найден/не найден
    alt
      else Пользователь найден
        auth -> otp: Отправить ОТП
        auth -> auth: Сохранить состояние(step, phone, OTP, user-id и др)
        auth -> frontend: OK\n*session-id\n*user-id\n*next_step =
AUTHORIZATION_OTP\n*OTP TTL
      else Пользователь не найден
        auth -> otp: Отправить ОТП
        auth -> auth: Сохранить состояние(step, phone, OTP и др)
        auth -> frontend: OK\n*session-id\n*next_step =
REGISTRATION_OTP\n*OTP TTL
    end
  end


  opt Пользователь новый (REGISTRATION_OTP)
    break
      frontend ->[:Регистрация\n(другой процесс)
    end break
  end opt

    frontend -> frontend: Форма ввода OTP
    frontend -> auth: Подтверждение OTP (авторизация), POST
/auth/verify\n*OTP code\n*session-id
    group Принять решение о вводе PIN
      auth -> auth: Получить данные по session id и проверить состояние
      auth -> auth: Проверить OTP

      auth -> auth: Проверить лимиты по попыткам ввода и тайминги

      auth -> auth: Сохранить состояние(step и др)
```

```
      auth -> frontend: OK\n*session-id\n*next_step = AUTHORIZATION_PIN

    end

    frontend -> frontend: Форма ввода PIN (создание)
    frontend -> frontend: Создать хэш по PIN, сохранить на устройстве
    frontend -> frontend: Подпись device id ключом device private key

    frontend -> auth: Завершить авторизацию, POST /auth/complete\n*session-
id\n*device public key\n*signed device id
    group Принять решение о выпуске токена
      auth -> auth: Получить данные по session id и проверить состояние

      auth -> auth: Инвалидировать session-id

      auth -> auth: Проверить подпись signed device id по device public
key\nпроверить device-id с данными в сессии

      auth -> device: Проверить наличие устройства по device-id
      alt
        else Устройства нет
          auth -> device: Создать устройство по данным сессии\nсохранить
device public key
        else Устройство есть
          auth -> auth: Проверить env data:\nизменились ли параметры
устройства и системы и как сильно
      end

      auth -> device: Привязать текущее устройство,\nсделать неактивными
ранее привязанные
      device -> auth: OK

      auth -> idp: Выпустить токены
      idp -> auth: access token, refresh token

      auth -> notification: (Опционально) Уведомление о входе с нового
устройства
      notification -> auth: OK

      auth -> frontend: OK\n*access token\n*refresh token\n*device-id\n*user-
id

    end

    frontend -> frontend: Выпуск ключа шифрования по PIN\n(IOS SecureEnclave)
    frontend -> frontend: Шифрование refresh token
    frontend -> frontend: Сохранить данные в защищенном хранилище\n-refresh
token (encrypted)\n*метаданные шифрования
    frontend -> frontend: Сохранить в памяти\n*access token
    frontend -> frontend: Сохранить метаданные\n*refresh token
expiration\n*device-id\n*user-id

@enduml
```


Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
notification
Запуск приложения
[01] Опредалить уникальный device id
собрать данные env data (устр-во, система, приложение)
[02] Принять решение о новой установке:
наличие device-key в локальном хранилище
opt [Новая установка приложения (нет device-key)]
[03] Сгенерировать device-key при его отсутствии,
сохранить private key + public key в хранилище
Принять решение об авторизации:
[04] • наличие user-id в локальном хранилище и отсутствует либо истек refresh token
• или отсутствие user-id в локальном хранилище
opt [Refresh token валидный и есть user-id]
break
[05] Упрощенная Авторизация
(другой процесс)
Полная авторизация пользователя
Старт процесса, POST /auth/init
[06] • device-id
• env data
Принять решение о сессии авторизации
[07] (Опционально) Оценка риска:
проверить версию приложения, IP, частоту запросов
[08] (Опционально) Оценка риска:
проверить black lists, данные по устройству, системе
[09] Создать сессию, session-id с TTL (10 мин?)
сохранить состояние (step, env data, device-id, проверки и др.)
OK
[10] • session-id
• session ttl
• next_step = IDENTIFICATION_PHONE
[11] Форма ввода номера телефона
Ввод телефона, POST /auth/phone
[12] -session-id
-phone
Принять решение о пользователе
[13] (Опционально) Оценка риска:
проверка black lists
[14] Найти пользователя по телефону
[15] Не найден/не найден
alt
[Пользователь найден]
[16] Отправить OTP
[17] Сохранить состояние(step, phone, OTP, user-id и др)
OK
[18] • session-id
• user-id
• next_step = AUTHORIZATION_OTP
• OTP TTL
[Пользователь не найден]
[19] Отправить OTP
[20] Сохранить состояние(step, phone, OTP и др)
OK
[21] • session-id
• next_step = REGISTRATION_OTP
• OTP TTL
opt [Пользователь новый (REGISTRATION_OTP)]
break
[22] Регистрация
(другой процесс)
[23] Форма ввода OTP
Подтверждение OTP (авторизация), POST /auth/verify
[24] • OTP code
• session-id
Принять решение о вводе PIN
[25] Получить данные по session id и проверить состояние
[26] Проверить OTP
[27] Проверить лимиты по попыткам ввода и таймиги
[28] Сохранить состояние(step и др)
OK
[29] • session-id
• next_step = AUTHORIZATION_PIN
[30] Форма ввода PIN (создание)
[31] Создать хэш по PIN, сохранить на устройстве
[32] Подпись device id ключом device private key
Завершить авторизацию, POST /auth/complete
[33] • session-id
• device public key
• signed device id
Принять решение о выпуске токена
[34] Получить данные по session id и проверить состояние
[35] Инвалидировать session-id
[36] Проверить подпись signed device id по device public key
проверить device-id с данными в сессии
[37] Проверить наличие устройства по device-id
alt
[Устройства нет]
[38] Создать устройство по данным сессии
сохранить device public key
[Устройство есть]
[39] Проверить env data:
изменились ли параметры устройства и системы и как сильно
[40] Привязать текущее устройство,
сделать неактивными ранее привязанные
[41] OK
[42] Выпустить токены
[43] access token, refresh token
[44] (Опционально) Уведомление о входе с нового устройства
[45] OK
OK
[46] • access token
• refresh token
• device-id
• user-id
[47] Выпуск ключа шифрования по PIN
(IOS SecureEnclave)
[48] Шифрование refresh token
Сохранить данные в защищенном хранилище
[49] -refresh token (encrypted)
• метаданные шифрования
[50] Сохранить в памяти
• access token

Регистрация. Предварительный процесс

Сценарии:

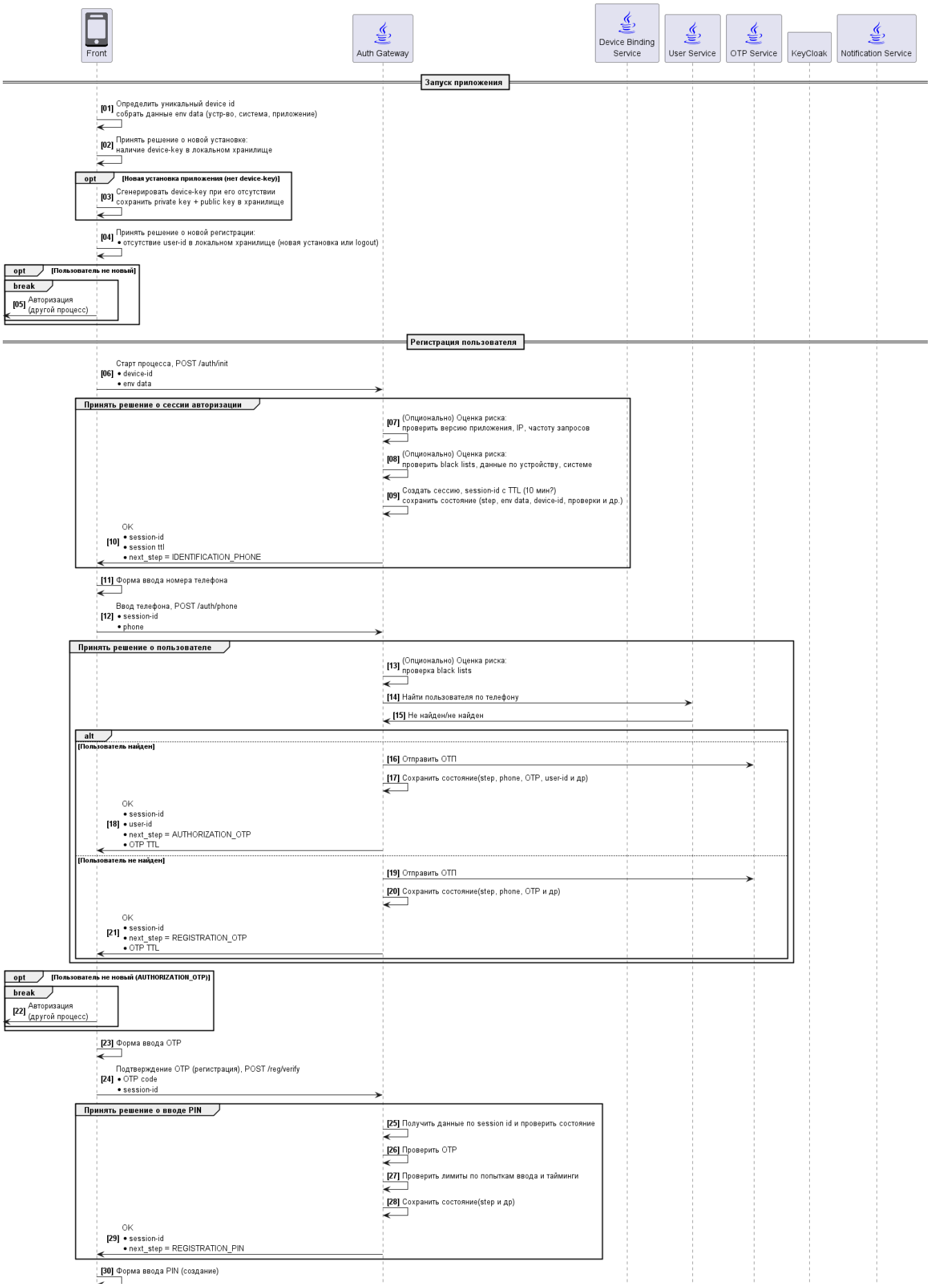
- первая установка приложения, пользователь не зарегистрирован
- регистрация второго пользователя в установленном приложении

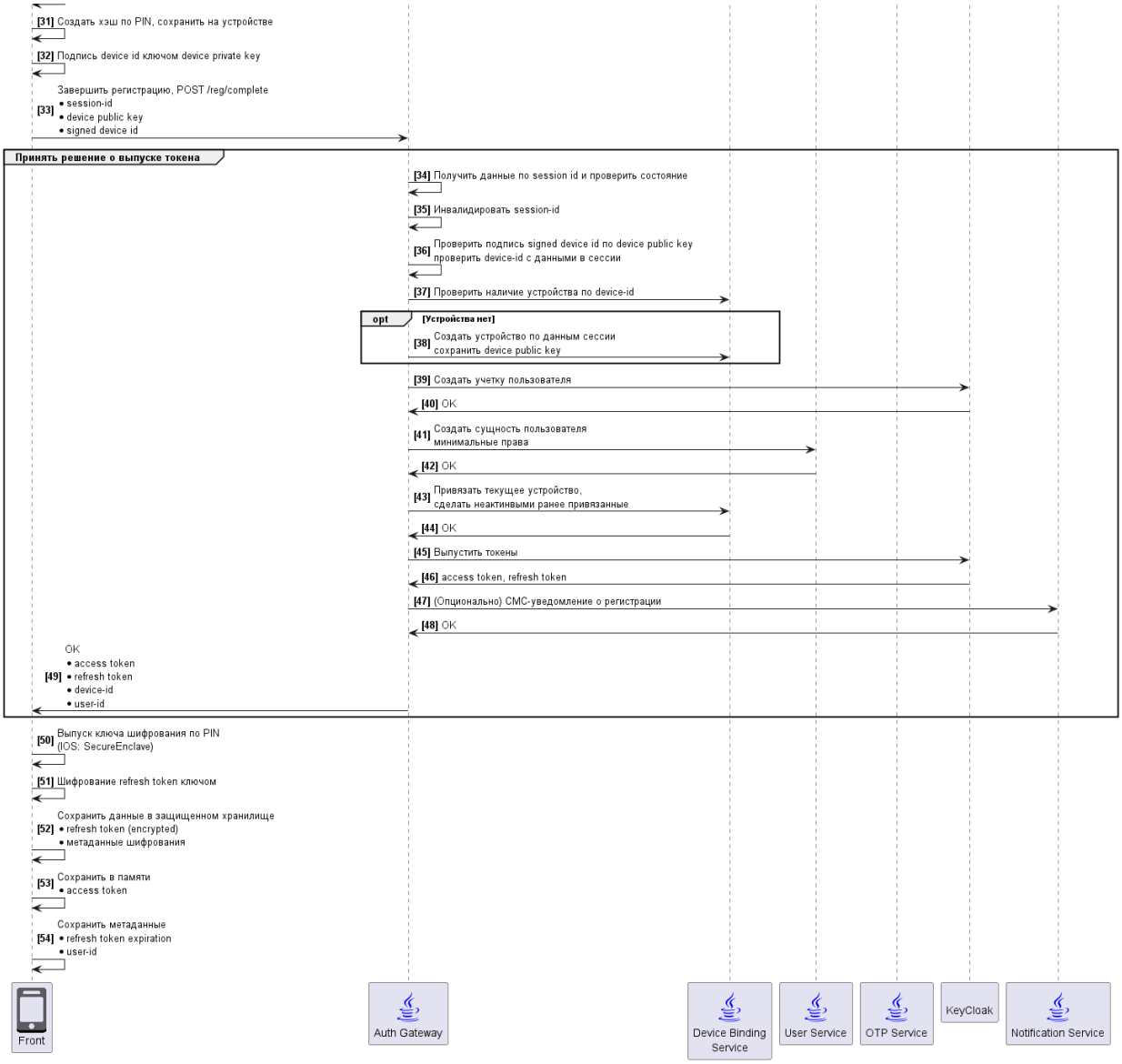
Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<font color=blue><$java>\nAuth Gateway" as auth
participant "<font color=blue><$java>\nDevice Binding\nService" as device
participant "<font color=blue><$java>\nUser Service" as user
participant "<font color=blue><$java>\nOTP Service" as otp
participant "<font color=blue>\nKeyCloak" as idp
participant "<font color=blue><$java>\nNotification Service" as notification


autonumber "<b>[00]"
==   ==

  frontend -> frontend:   device id\n  env data (-, , )

  frontend -> frontend:     :\n device-key

  opt    ( device-key)
    frontend -> frontend:  device-key   \n private key + public key
  end opt

  frontend -> frontend:     :\n* user-id    (   logout)

  opt
    break
      frontend ->[:\n( )
    end break
  end opt

==   ==
  frontend -> auth:  , POST /auth/init\n*device-id\n*env data
  group
    auth -> auth: ()  : \n  , IP,
    auth -> auth: ()  : \n black lists,   ,

    auth -> auth:  , session-id  TTL (10 ?)\n  (step, env data, device-id,   .)

    auth -> frontend: OK\n*session-id\n*session ttl\n*next_step = IDENTIFICATION_PHONE
  end

  frontend -> frontend:
  frontend -> auth:  , POST /auth/phone\n*session-id\n*phone
  group

    auth -> auth: ()  :\n black lists

    auth -> user:
    user -> auth:  /
    alt
      else
```

```
      auth -> otp:
      auth -> auth:  (step, phone, OTP, user-id  )
      auth -> frontend: OK\n*session-id\n*user-id\n*next_step = AUTHORIZATION_OTP\n*OTP TTL
    else
      auth -> otp:
      auth -> auth:  (step, phone, OTP  )
      auth -> frontend: OK\n*session-id\n*next_step = REGISTRATION_OTP\n*OTP TTL
  end
end

opt    (AUTHORIZATION_OTP)
  break
    frontend ->[:\n( )
  end break
end opt

  frontend -> frontend:   OTP
  frontend -> auth:  OTP (), POST /reg/verify\n*OTP code\n*session-id
  group     PIN
    auth -> auth:    session id
    auth -> auth:  OTP

    auth -> auth:

    auth -> auth:  (step  )

    auth -> frontend: OK\n*session-id\n*next_step = REGISTRATION_PIN

  end

  frontend -> frontend:   PIN ()
  frontend -> frontend:    PIN,
  frontend -> frontend:  device id  device private key

  frontend -> auth:  , POST /reg/complete\n*session-id\n*device public key\n*signed device id
  group
    auth -> auth:    session id

    auth -> auth:  session-id

    auth -> auth:   signed device id  device public key\n device-id

    auth -> device:     device-id
    opt
      auth -> device:     \n device public key
    end

    auth -> idp:
    idp -> auth: OK

    auth -> user:   \n
    user -> auth: OK

    auth -> device:   ,\n
    device -> auth: OK

    auth -> idp:
    idp -> auth: access token, refresh token

    auth -> notification: () -
    notification -> auth: OK

    auth -> frontend: OK\n*access token\n*refresh token\n*device-id\n*user-id

  end

  frontend -> frontend:     PIN\n(IOS: SecureEnclave)
  frontend -> frontend:  refresh token
  frontend -> frontend:     \n*refresh token (encrypted)\n*
  frontend -> frontend:   \n*access token
  frontend -> frontend:  \n*refresh token expiration\n*user-id
```

```
@enduml
```


Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
Notification Service
Запуск приложения
[01] Определить уникальный device id
собрать данные env data (устр-во, система, приложение)
[02] Принять решение о новой установке:
наличие device-key в локальном хранилище
opt [Новая установка приложения (нет device-key)]
[03] Сгенерировать device-key при его отсутствии
сохранить private key + public key в хранилище
[04] Принять решение о новой регистрации:
• отсутствие user-id в локальном хранилище (новая установка или logout)
opt [Пользователь не новый]
break
[05] Авторизация
(другой процесс)
Регистрация пользователя
Старт процесса, POST /auth/init
[06] • device-id
• env data
Принять решение о сессии авторизации
[07] (Опционально) Оценка риска:
проверить версию приложения, IP, частоту запросов
[08] (Опционально) Оценка риска:
проверить black lists, данные по устройству, системе
[09] Создать сессию, session-id с TTL (10 мин?)
сохранить состояние (step, env data, device-id, проверки и др.)
OK
[10] • session-id
• session ttl
• next_step = IDENTIFICATION_PHONE
[11] Форма ввода номера телефона
Ввод телефона, POST /auth/phone
[12] • session-id
• phone
Принять решение о пользователе
[13] (Опционально) Оценка риска:
проверка black lists
[14] Найти пользователя по телефону
[15] Не найден/не найден
alt
[Пользователь найден]
[16] Отправить ОТП
[17] Сохранить состояние(step, phone, OTP, user-id и др)
OK
[18] • session-id
• user-id
• next_step = AUTHORIZATION_OTP
• OTP TTL
[Пользователь не найден]
[19] Отправить ОТП
[20] Сохранить состояние(step, phone, OTP и др)
OK
[21] • session-id
• next_step = REGISTRATION_OTP
• OTP TTL
opt [Пользователь не новый (AUTHORIZATION_OTP)]
break
[22] Авторизация
(другой процесс)
[23] Форма ввода OTP
Подтверждение OTP (регистрация), POST /reg/verify
[24] • OTP code
• session-id
Принять решение о вводе PIN
[25] Получить данные по session id и проверить состояние
[26] Проверить OTP
[27] Проверить лимиты по попыткам ввода и тайминги
[28] Сохранить состояние(step и др)
OK
[29] • session-id
• next_step = REGISTRATION_PIN
[30] Форма ввода PIN (создание)


[31] Создать хэш по PIN, сохранить на устройстве
[32] Подпись device id ключом device private key
[33] Завершить регистрацию, POST /reg/complete
• session-id
• device public key
• signed device id
Принять решение о выпуске токена
[34] Получить данные по session id и проверить состояние
[35] Инвалидировать session-id
[36] Проверить подпись signed device id по device public key
проверить device-id с данными в сессии
[37] Проверить наличие устройства по device-id
opt
[Устройства нет]
[38] Создать устройство по данным сессии
сохранить device public key
[39] Создать учетку пользователя
[40] OK
[41] Создать сущность пользователя
минимальные права
[42] OK
[43] Привязать текущее устройство,
сделать неактивными ранее привязанные
[44] OK
[45] Выпустить токены
[46] access token, refresh token
[47] (Опционально) СМС-уведомление о регистрации
[48] OK
[49] OK
• access token
• refresh token
• device-id
• user-id
[50] Выпуск ключа шифрования по PIN
(IOS: SecureEnclave)
[51] Шифрование refresh token ключом
[52] Сохранить данные в защищенном хранилище
• refresh token (encrypted)
• метаданные шифрования
[53] Сохранить в памяти
• access token
[54] Сохранить метаданные
• refresh token expiration
• user-id
Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
Notification Service

Регистрация. Предварительный процесс

Сценарии:

- первая установка приложения, пользователь не зарегистрирован
- регистрация второго пользователя в установленном приложении

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<font color=blue><$java>\nAuth Gateway" as auth
participant "<font color=blue><$java>\nDevice Binding\nService" as device
participant "<font color=blue><$java>\nUser Service" as user
participant "<font color=blue><$java>\nOTP Service" as otp
participant "<font color=blue>\nKeyCloak" as idp
participant "<font color=blue><$java>\nNotification Service" as notification


autonumber "<b>[00]"
== Запуск приложения ==

  frontend -> frontend: Определить уникальный device id\nсобрать данные env
data (устр-во, система, приложение)

  frontend -> frontend: Принять решение о новой установке:\nналичие device-
key в локальном хранилище

  opt Новая установка приложения (нет device-key)
    frontend -> frontend: Сгенерировать device-key при его
отсутствии\nсохранить private key + public key в хранилище
  end opt

  frontend -> frontend: Принять решение о новой регистрации:\n*отсутствие
user-id в локальном хранилище (новая установка или logout)

  opt Пользователь не новый
    break
      frontend ->[:Авторизация\n(другой процесс)
    end break
  end opt

== Регистрация пользователя ==
```

```
  frontend -> auth: Старт процесса, POST /auth/init\n*device-id\n*env data
  group Принять решение о сессии авторизации
    auth -> auth: (Опционально) Оценка риска: \nпроверить версию приложения,
IP, частоту запросов
    auth -> auth: (Опционально) Оценка риска: \nпроверить black lists, данные
по устройству, системе

    auth -> auth: Создать сессию, session-id с TTL (10 мин?)\nсохранить
состояние (step, env data, device-id, проверки и др.)

    auth -> frontend: OK\n*session-id\n*session ttl\n*next_step =
IDENTIFICATION_PHONE
  end

  frontend -> frontend: Форма ввода номера телефона
  frontend -> auth: Ввод телефона, POST /auth/phone\n*session-id\n*phone
  group Принять решение о пользователе

    auth -> auth: (Опционально) Оценка риска:\nпроверка black lists

    auth -> user: Найти пользователя по телефону
    user -> auth: Не найден/не найден
    alt
      else Пользователь найден
        auth -> otp: Отправить ОТП
        auth -> auth: Сохранить состояние(step, phone, OTP, user-id и др)
        auth -> frontend: OK\n*session-id\n*user-id\n*next_step =
AUTHORIZATION_OTP\n*OTP TTL
      else Пользователь не найден
        auth -> otp: Отправить ОТП
        auth -> auth: Сохранить состояние(step, phone, OTP и др)
        auth -> frontend: OK\n*session-id\n*next_step =
REGISTRATION_OTP\n*OTP TTL
    end
  end

  opt Пользователь не новый (AUTHORIZATION_OTP)
    break
      frontend ->[:Авторизация\n(другой процесс)
    end break
  end opt

    frontend -> frontend: Форма ввода OTP
    frontend -> auth: Подтверждение OTP (регистрация), POST /reg/verify\n*OTP
code\n*session-id
    group Принять решение о вводе PIN
      auth -> auth: Получить данные по session id и проверить состояние
      auth -> auth: Проверить OTP

      auth -> auth: Проверить лимиты по попыткам ввода и тайминги

      auth -> auth: Сохранить состояние(step и др)

      auth -> frontend: OK\n*session-id\n*next_step = REGISTRATION_PIN

    end
```

```
    frontend -> frontend: Форма ввода PIN (создание)
    frontend -> frontend: Создать хэш по PIN, сохранить на устройстве
    frontend -> frontend: Подпись device id ключом device private key

    frontend -> auth: Завершить регистрацию, POST /reg/complete\n*session-
id\n*device public key\n*signed device id
    group Принять решение о выпуске токена
      auth -> auth: Получить данные по session id и проверить состояние

      auth -> auth: Инвалидировать session-id

      auth -> auth: Проверить подпись signed device id по device public
key\nпроверить device-id с данными в сессии

      auth -> device: Проверить наличие устройства по device-id
      opt Устройства нет
        auth -> device: Создать устройство по данным сессии\nсохранить device
public key
      end

      auth -> idp: Создать учетку пользователя
      idp -> auth: OK

      auth -> user: Создать сущность пользователя\nминимальные права
      user -> auth: OK

      auth -> device: Привязать текущее устройство,\nсделать неактинвыми
ранее привязанные
      device -> auth: OK

      auth -> idp: Выпустить токены
      idp -> auth: access token, refresh token

      auth -> notification: (Опционально) СМС-уведомление о регистрации
      notification -> auth: OK

      auth -> frontend: OK\n*access token\n*refresh token\n*device-id\n*user-
id

    end

    frontend -> frontend: Выпуск ключа шифрования по PIN\n(IOS:
SecureEnclave)
    frontend -> frontend: Шифрование refresh token ключом
    frontend -> frontend: Сохранить данные в защищенном хранилище\n*refresh
token (encrypted)\n*метаданные шифрования
    frontend -> frontend: Сохранить в памяти\n*access token
    frontend -> frontend: Сохранить метаданные\n*refresh token
expiration\n*user-id

@enduml
```

Front
Auth Gateway
Device Binding Service
User Service
OTP Service
KeyCloak
Notification Service
Запуск приложения
[01] Определить уникальный device id
собрать данные env data (устр-во, система, приложение)
[02] Принять решение о новой установке:
наличие device-key в локальном хранилище
opt [Новая установка приложения (нет device-key)]
[03] Сгенерировать device-key при его отсутствии
сохранить private key + public key в хранилище
[04] Принять решение о новой регистрации:
• отсутствие user-id в локальном хранилище (новая установка или logout)
opt [Пользователь не новый]
break
[05] Авторизация
(другой процесс)
Регистрация пользователя
Старт процесса, POST /auth/init
[06] • device-id
• env data
Принять решение о сессии авторизации
[07] (Опционально) Оценка риска:
проверить версию приложения, IP, частоту запросов
[08] (Опционально) Оценка риска:
проверить black lists, данные по устройству, системе
[09] Создать сессию, session-id с TTL (10 мин?)
сохранить состояние (step, env data, device-id, проверки и др.)
OK
[10] • session-id
• session ttl
• next_step = IDENTIFICATION_PHONE
[11] Форма ввода номера телефона
Ввод телефона, POST /auth/phone
[12] • session-id
• phone
Принять решение о пользователе
[13] (Опционально) Оценка риска:
проверка black lists
[14] Найти пользователя по телефону
[15] Не найден/не найден
alt
[Пользователь найден]
[16] Отправить OTP
[17] Сохранить состояние(step, phone, OTP, user-id и др)
OK
[18] • session-id
• user-id
• next_step = AUTHORIZATION_OTP
• OTP TTL
[Пользователь не найден]
[19] Отправить OTP
[20] Сохранить состояние(step, phone, OTP и др)
OK
[21] • session-id
• next_step = REGISTRATION_OTP
• OTP TTL
opt [Пользователь не новый (AUTHORIZATION_OTP)]
break
[22] Авторизация
(другой процесс)
[23] Форма ввода OTP
Подтверждение OTP (регистрация), POST /reg/verify
[24] • OTP code
• session-id
Принять решение о вводе PIN
[25] Получить данные по session id и проверить состояние
[26] Проверить OTP
[27] Проверить лимиты по попыткам ввода и таймиги
[28] Сохранить состояние(step и др)
OK
[29] • session-id
• next_step = REGISTRATION_PIN
[30] Форма ввода PIN (создание)
[31] Создать хэш по PIN, сохранить на устройстве
[32] Подпись device id ключом device private key
Завершить регистрацию, POST /reg/complete
[33] • session-id
• device public key
• signed device id
Принять решение о выпуске токена
[34] Получить данные по session id и проверить состояние
[35] Инвалидировать session-id
[36] Проверить подпись signed device id по device public key
проверить device-id с данными в сессии
[37] Проверить наличие устройства по device-id
opt [Устройства нет]
[38] Создать устройство по данным сессии
сохранить device public key
[39] Создать учетку пользователя
[40] OK
[41] Создать сущность пользователя
минимальные права
[42] OK
[43] Привязать текущее устройство,
сделать неактивными ранее привязанные
[44] OK
[45] Выпустить токены
[46] access token, refresh token
[47] (Опционально) СМС-уведомление о регистрации
[48] OK
OK
[49] • access token
• refresh token
• device-id
• user-id
[50] Выпуск ключа шифрования по PIN
(IOS: SecureEnclave)
[51] Шифрование refresh token ключом
Сохранить данные в защищенном хранилище

Схема взаимодействия [центробанк ГЦИ]

Предварительные шаги:

- создать банковский сертификат в Vault, под которым будут выдаваться клиентские сертификаты - intermediate CA
    - можно исопльзовать утилиту от ЦБ и импортировать в Vault либо средствами Vault
- настроить Kong на работу с mTLS для сервиса GCI Adapter, импортировать public key банковского сертификата (используется Kong для проверки подписи клиентских сертификатов).
- настроить в Rong АПИ для проксирования запроса в ГЦИ, настроить определение пути обращения и заполнения header
- (под вопросом) создать отдельную пару ключей private / public для подписи JWT-токена - access token, передать public key в ГЦИ для последующей проверки подписи JWT
    - как вариант - выставить jwks ендпоинт с public key

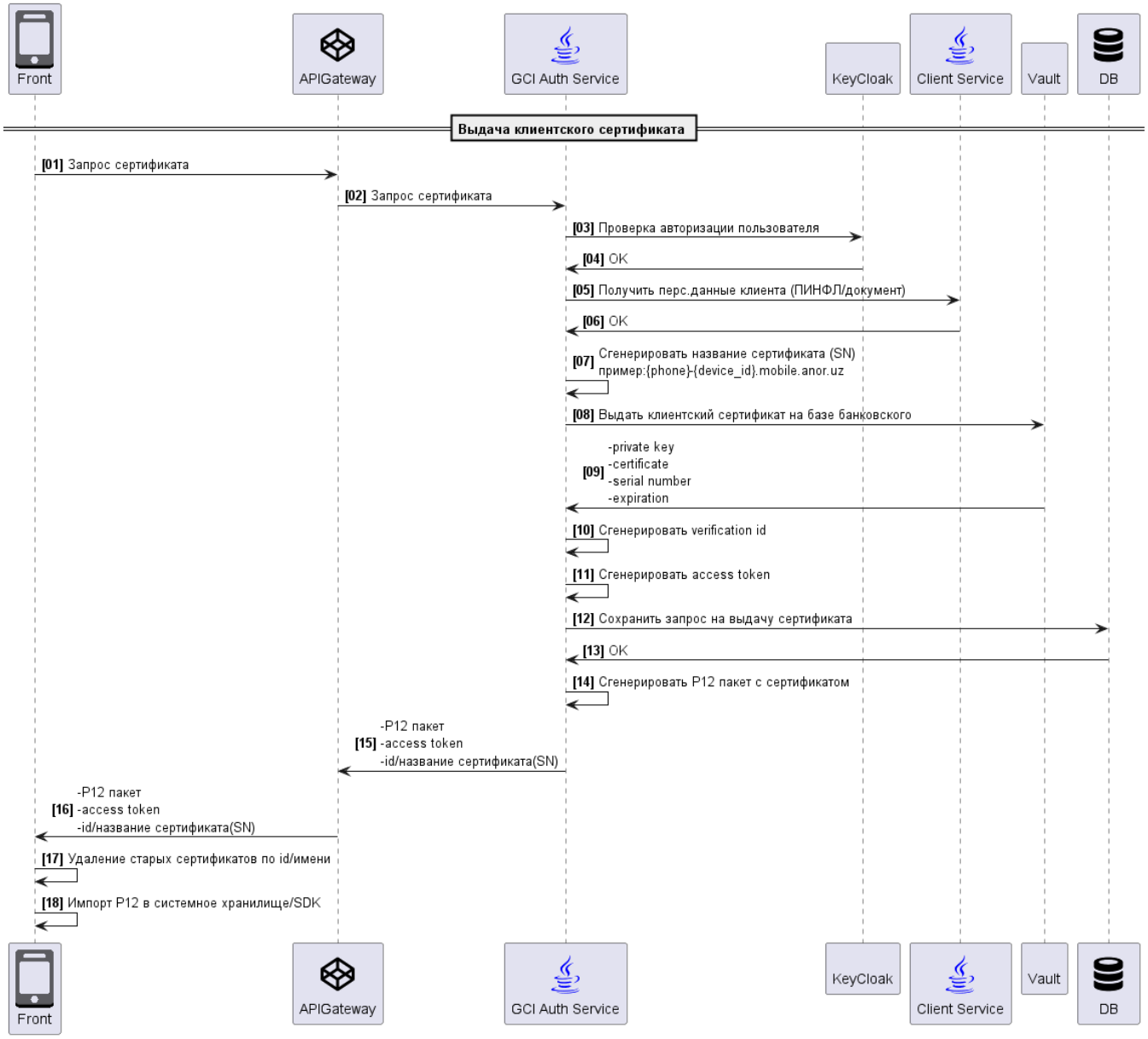
Подход к проксированию:

- проксируем вслепую, т.е. ничего не знаем о структуре запроса, ответа и внутреннем пути АПИ ГЦИ. Знаем только хост ГЦИ
- на фронте для SDK задаем базовый путь банка для запросов в виде http://anorbank.uz/api/v1/identity/verification этот же путь мы заводим в Kong как АПИ
- SDK шлет запрос на адрес: базовый путь + ендпоинт ГЦИ (например, /api/v1/gsi/verify). Получается адрес http://anorbank.uz/api/v1/identity/verification/api/v1/gsi/verify
- Kong для настроенного АПИ (http://anorbank.uz/api/v1/identity/verification) настраивается так, чтобы перенаправить запрос до сервиса бэка и остаток пути обернуть в header. Получается запрос (АПИ уже бэк сервиса gci) http://gci-adapter/api/v1/identity/verification, header X-Gci-Path: /api/v1/gsi/verify
- сервис проксирует запрос до хоста ЦБ, добавляя путь из X-Gci-Path: https://face.mbabm.uz/api/v1/gsi/verify

Запрос на выдачу сертификата (предварительно)

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<$codepen>\nAPIGateway" as gw
participant "<font color=blue><$java>\nGCI Auth Service" as auth
participant "<font color=blue>\nKeyCloak" as idp
participant "<font color=blue><$java>\nClient Service" as client
participant "<font color=blue>\nVault" as vault
participant "<$database>\nDB" as db

autonumber "<b>[00]"
==    ==

  frontend -> gw:
  gw -> auth:

  auth -> idp:
  idp -> auth: OK

  auth -> client:  .  (/)
  client -> auth: OK

  auth-> auth:    (SN)\n:{phone}-{device_id}.mobile.anor.uz

  auth -> vault:
  vault -> auth: -private key\n-certificate\n-serial number\n-expiration

  auth -> auth:  verification id
  auth -> auth:  access token

  auth -> db:
  db -> auth: OK

  auth -> auth:  P12

  auth -> gw: -P12 \n-access token\n-id/ (SN)

  gw -> frontend: -P12 \n-access token\n-id/ (SN)

  frontend -> frontend:     id/
  frontend -> frontend:  P12   /SDK

@enduml
```



Запрос на проверку биометрии (предварительно)

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <cloudinsight/kafka>

participant "<$mobileclient>\nFront\nSDK" as frontend
participant "<$codepen>\nAPIGateway" as gw
participant "<font color=blue><$java>\nGCI Adapter" as adapter
participant "<font color=blue><$java>\nGCI Auth Service" as auth
participant "<font color=blue>\nVault" as vault
participant "<$database>\nDB" as db
participant "<$kafka>Kafka" as kafka

box \n
participant "<font color=blue>\n" as gci
end box

autonumber "<b>[00]"
==   ==

  frontend -> gw:  \nhttp://anorbank.uz/api/v1/identity/verification\nAuthorization:  access token\n-
  gw -> gw:   access token ()
  gw -> gw:

  gw -> adapter:  \nhttp://gci-adapter/api/v1/identity/verification\nAuthorization:  access token\nX-Gci-Path:
/api/v1/gsi/verify\n-\n- access token

  adapter -> auth:     \n( verification id  access token)
   auth -> db:
   db -> auth: OK
  auth -> adapter: OK

  adapter -> gci:     \nhttps://face.mbabm.uz/api/v1/gsi/verify\nAuthorization:  access token\n-
  gci -> adapter:

  adapter -> adapter:

  adapter -> auth:
    auth -> vault:  \n(Certificate Revocation List)
    vault -> auth: OK
  auth -> adapter: OK

  adapter -> kafka:     \n(    )

  adapter -> gw:   ( )

  gw -> frontend:   ( )

@enduml
```

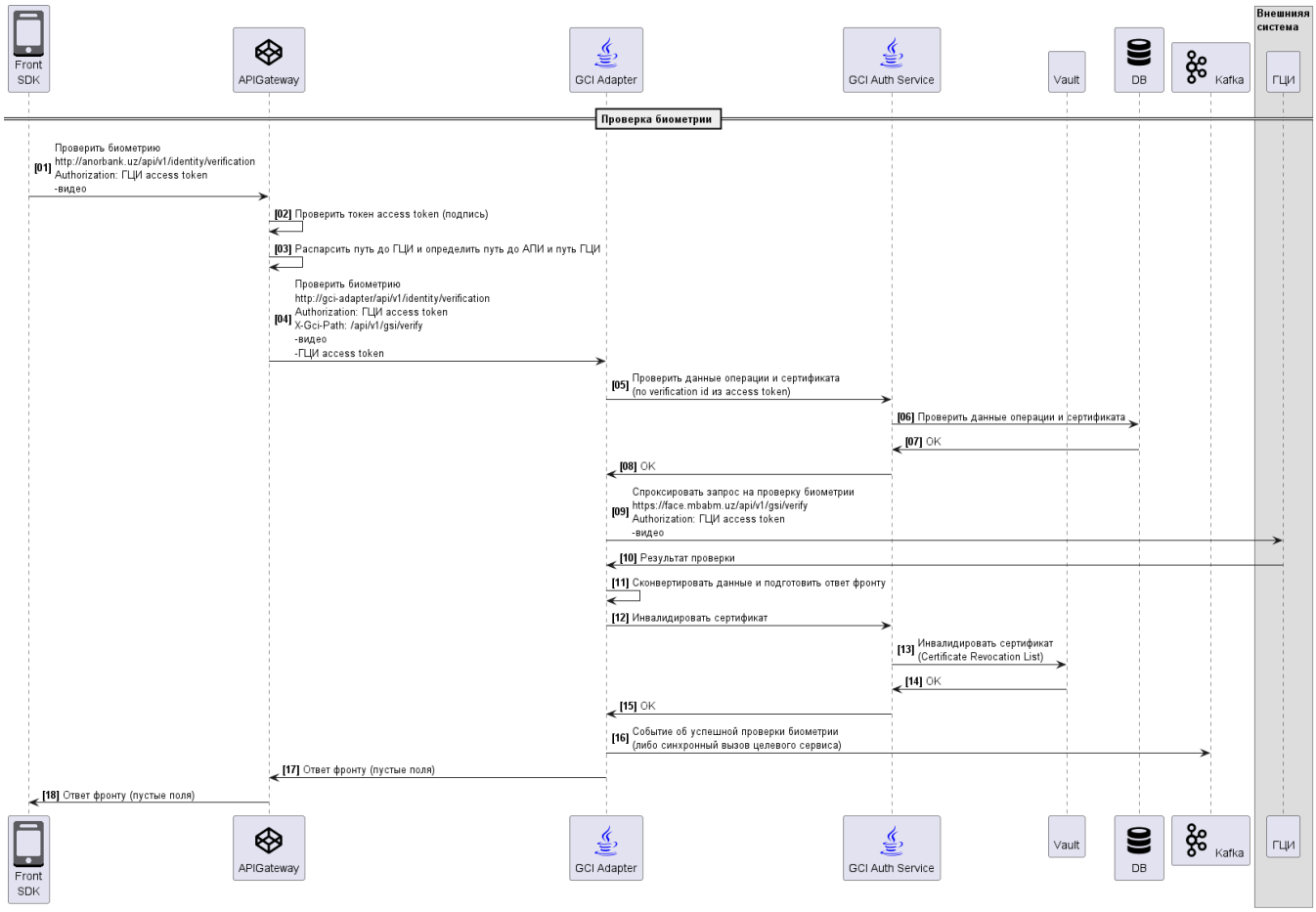

Front SDK
APIGateway
GCI Adapter
GCI Auth Service
Vault
DB
Kafka
Внешняя система
ГЦИ
Проверка биометрии
[01] Проверить биометрию
http://anorbank.uz/api/v1/identity/verification
Authorization: ГЦИ access token
-видео
[02] Проверить токен access token (подпись)
[03] Распарсить путь до ГЦИ и определить путь до АПИ и путь ГЦИ
[04] Проверить биометрию
http://gci-adapter/api/v1/identity/verification
Authorization: ГЦИ access token
X-Gci-Path: /api/v1/gsi/verify
-видео
-ГЦИ access token
[05] Проверить данные операции и сертификата
(по verification id из access token)
[06] Проверить данные операции и сертификата
[07] OK
[08] OK
[09] Спроксировать запрос на проверку биометрии
https://face.mbabm.uz/api/v1/gsi/verify
Authorization: ГЦИ access token
-видео
[10] Результат проверки
[11] Сконвертировать данные и подготовить ответ фронту
[12] Инвалидировать сертификат
[13] Инвалидировать сертификат
(Certificate Revocation List)
[14] OK
[15] OK
[16] Событие об успешной проверки биометрии
(либо синхронный вызов целевого сервиса)
[17] Ответ фронту (пустые поля)
[18] Ответ фронту (пустые поля)

Схема взаимодействия [центробанк ГЦИ]

Предварительные шаги:

- создать банковский сертификат в Vault, под которым будут выдаваться клиентские сертификаты - intermediate CA
    - можно исопльзовать утилиту от ЦБ и импортировать в Vault либо средствами Vault
- настроить Kong на работу с mTLS для сервиса GCI Adapter, импортировать public key банковского сертификата (используется Kong для проверки подписи клиентских сертификатов).
- настроить в Rong АПИ для проксирования запроса в ГЦИ, настроить определение пути обращения и заполнения header
- (под вопросом) создать отдельную пару ключей private / public для подписи JWT-токена - access token, передать public key в ГЦИ для последующей проверки подписи JWT
    - как вариант - выставить jwks ендпоинт с public key

Подход к проксированию:

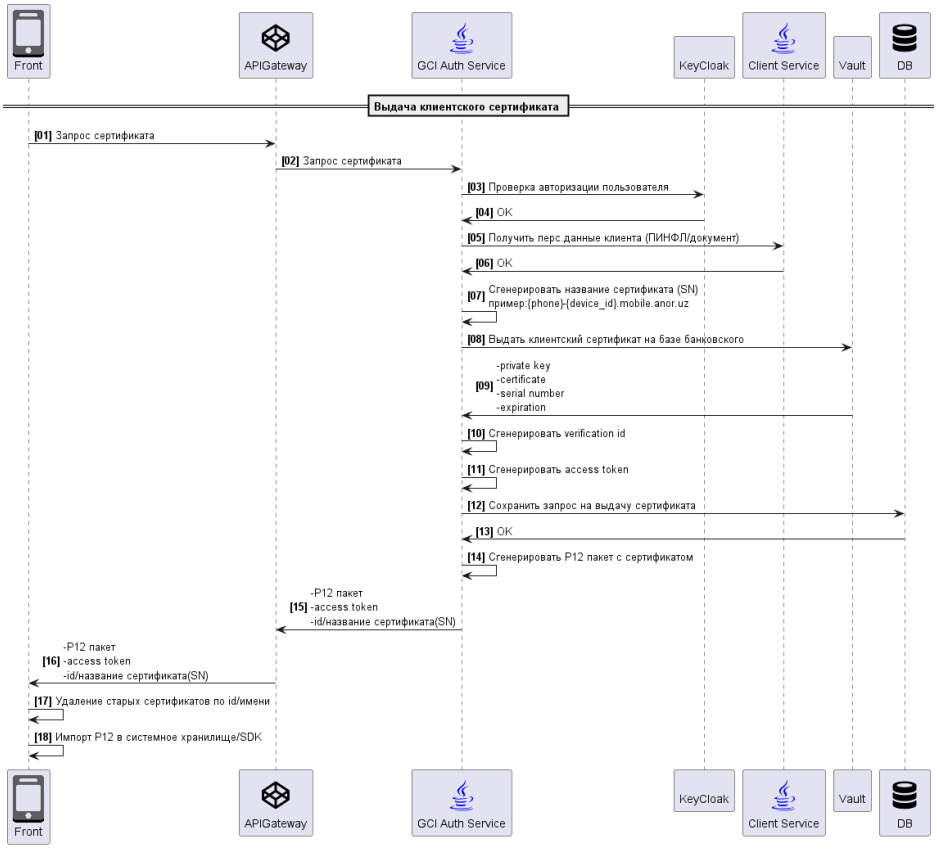
- проксируем вслепую, т.е. ничего не знаем о структуре запроса, ответа и внутреннем пути АПИ ГЦИ. Знаем только хост ГЦИ
- на фронте для SDK задаем базовый путь банка для запросов в виде http://anorbank.uz/api/v1/identity/verification этот же путь мы заводим в Kong как АПИ
- SDK шлет запрос на адрес: базовый путь + ендпоинт ГЦИ (например, /api/v1/gsi/verify). Получается адрес http://anorbank.uz/api/v1/identity/verification/api/v1/gsi/verify
- Kong для настроенного АПИ (http://anorbank.uz/api/v1/identity/verification) настраивается так, чтобы перенаправить запрос до сервиса бэка и остаток пути обернуть в header. Получается запрос (АПИ уже бэк сервиса gci) http://gci-adapter/api/v1/identity/verification, header X-Gci-Path: /api/v1/gsi/verify
- сервис проксирует запрос до хоста ЦБ, добавляя путь из X-Gci-Path: https://face.mbabm.uz/api/v1/gsi/verify

Запрос на выдачу сертификата (предварительно)

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
```

```
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<$codepen>\nAPIGateway" as gw
participant "<font color=blue><$java>\nGCI Auth Service" as auth
participant "<font color=blue>\nKeyCloak" as idp
participant "<font color=blue><$java>\nClient Service" as client
participant "<font color=blue>\nVault" as vault
participant "<$database>\nDB" as db

autonumber "<b>[00]"
== Выдача клиентского сертификата ==

  frontend -> gw: Запрос сертификата
  gw -> auth: Запрос сертификата

  auth -> idp: Проверка авторизации пользователя
  idp -> auth: OK

  auth -> client: Получить перс.данные клиента (ПИНФЛ/документ)
  client -> auth: OK

  auth-> auth: Сгенерировать название сертификата (SN)\nпример:{phone}-
{device_id}.mobile.anor.uz

  auth -> vault: Выдать клиентский сертификат на базе банковского
  vault -> auth: -private key\n-certificate\n-serial number\n-expiration

  auth -> auth: Сгенерировать verification id
  auth -> auth: Сгенерировать access token

  auth -> db: Сохранить запрос на выдачу сертификата
  db -> auth: OK

  auth -> auth: Сгенерировать P12 пакет с сертификатом

  auth -> gw: -P12 пакет\n-access token\n-id/название сертификата(SN)

  gw -> frontend: -P12 пакет\n-access token\n-id/название сертификата(SN)

  frontend -> frontend: Удаление старых сертификатов по id/имени
  frontend -> frontend: Импорт P12 в системное хранилище/SDK

@enduml
```

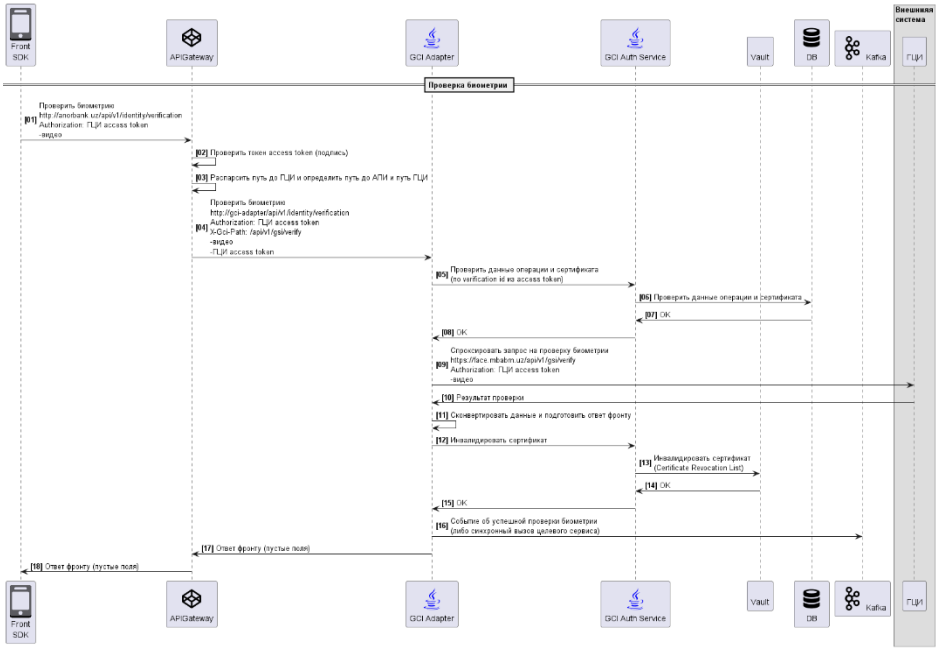


Запрос на проверку биометрии (предварительно)

Plant UML

```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <cloudinsight/kafka>

participant "<$mobileclient>\nFront\nSDK" as frontend
participant "<$codepen>\nAPIGateway" as gw
participant "<font color=blue><$java>\nGCI Adapter" as adapter
participant "<font color=blue><$java>\nGCI Auth Service" as auth
participant "<font color=blue>\nVault" as vault
participant "<$database>\nDB" as db
participant "<$kafka>Kafka" as kafka

box Внешнияя\nсистема
participant "<font color=blue>\nГЦИ" as gci
end box

autonumber "<b>[00]"
== Проверка биометрии ==
```

```
  frontend -> gw: Проверить
биометрию\nhttp://anorbank.uz/api/v1/identity/verification\nAuthorization:
ГЦИ access token\n-видео
  gw -> gw: Проверить токен access token (подпись)
  gw -> gw: Распарсить путь до ГЦИ и определить путь до АПИ и путь ГЦИ

  gw -> adapter: Проверить биометрию\nhttp://gci-
adapter/api/v1/identity/verification\nAuthorization: ГЦИ access token\nX-Gci-
Path: /api/v1/gsi/verify\n-видео\n-ГЦИ access token

  adapter -> auth: Проверить данные операции и сертификата\n(по verification
id из access token)
   auth -> db: Проверить данные операции и сертификата
   db -> auth: OK
  auth -> adapter: OK

  adapter -> gci: Спроксировать запрос на проверку
биометрии\nhttps://face.mbabm.uz/api/v1/gsi/verify\nAuthorization: ГЦИ access
token\n-видео
  gci -> adapter: Результат проверки

  adapter -> adapter: Сконвертировать данные и подготовить ответ фронту

  adapter -> auth: Инвалидировать сертификат
    auth -> vault: Инвалидировать сертификат\n(Certificate Revocation List)
    vault -> auth: OK
  auth -> adapter: OK

  adapter -> kafka: Событие об успешной проверки биометрии\n(либо синхронный
вызов целевого сервиса)

  adapter -> gw: Ответ фронту (пустые поля)

  gw -> frontend: Ответ фронту (пустые поля)

@enduml
```


Внешняя система
Front SDK
APIGateway
GCI Adapter
GCI Auth Service
Vault
DB
Kafka
ГЦИ
Проверка биометрии
[01] Проверить биометрию
http://mobank.uz/api/v1/identity/verification
Authorization: ГЦИ access token
-видео
[02] Проверить токен access token (подпись)
[03] Распарсить путь до ГЦИ и определить путь до АПИ и путь ГЦИ
[04] Проверить биометрию
http://gci-adapter/api/v1/identity/verification
Authorization: ГЦИ access token
X-Gci-Path: /api/v1/gsi/verify
-видео
-ГЦИ access token
[05] Проверить данные операции и сертификата
(no verification id из access token)
[06] Проверить данные операции и сертификата
[07] OK
[08] OK
[09] Спроксировать запрос на проверку биометрии
https://face.mbabm.uz/api/v1/gsi/verify
Authorization: ГЦИ access token
-видео
[10] Результат проверки
[11] Сконвертировать данные и подготовить ответ фронту
[12] Инвалидировать сертификат
[13] Инвалидировать сертификат
(Certificate Revocation List)
[14] OK
[15] OK
[16] Событие об успешной проверки биометрии
(либо синхронный вызов целевого сервиса)
[17] Ответ фронту (пустые поля)
[18] Ответ фронту (пустые поля)

Упрощенная авторизация. Предварительный процесс

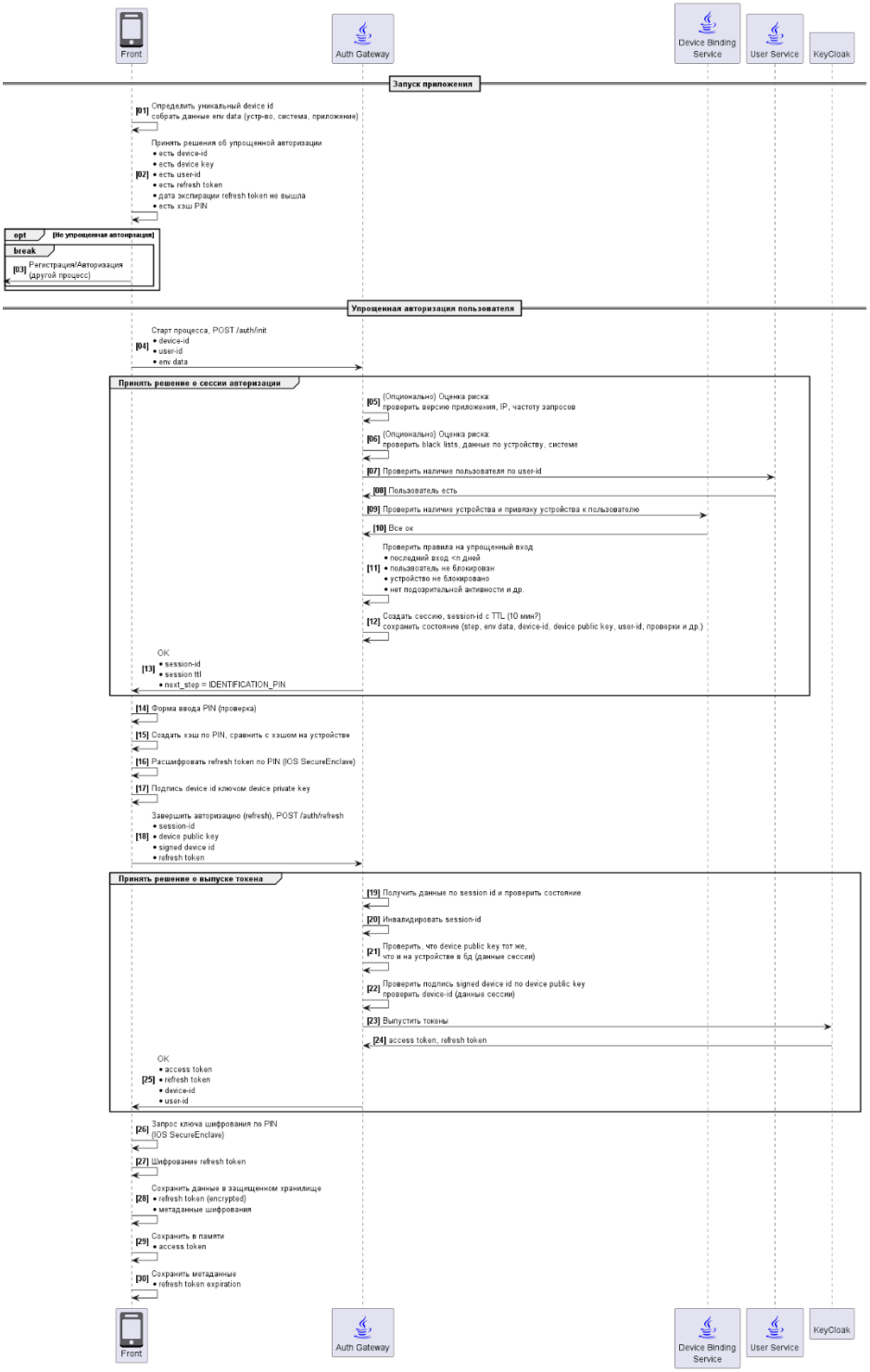
Сценарии:

- авторизация пользователя в установленном приложении, пользователь уже входил ранее (смены пользователей не было)

Plant UML

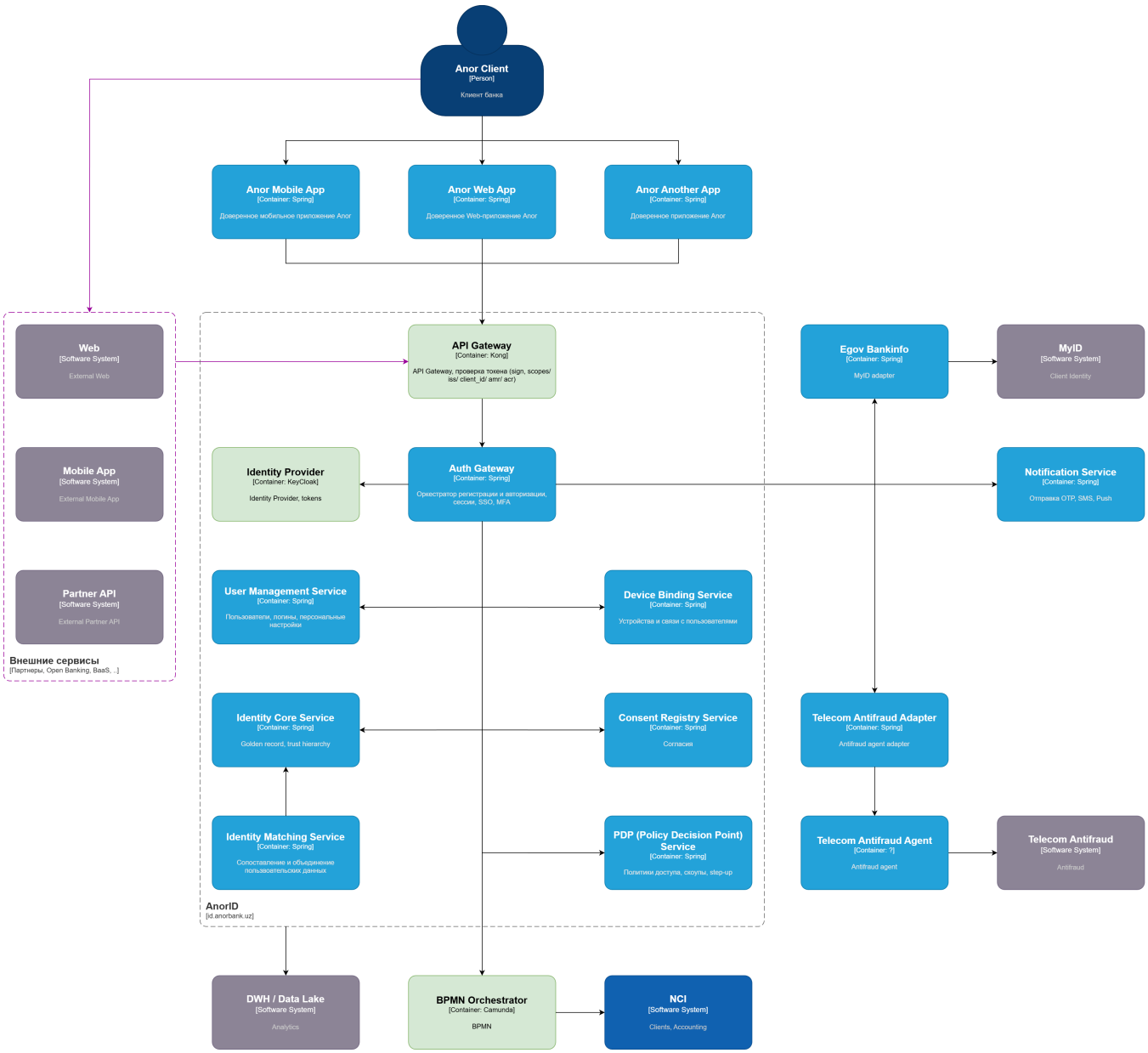
```
@startuml

!include <tupadr3/devicons/java>
!include <aws/General/mobileclient/mobileclient>
!include <tupadr3/devicons/database>
!include <tupadr3/devicons/codepen>
!include <tupadr3/devicons/redis>

participant "<$mobileclient>\nFront" as frontend
participant "<font color=blue><$java>\nAuth Gateway" as auth
participant "<font color=blue><$java>\nDevice Binding\nService" as device
participant "<font color=blue><$java>\nUser Service" as user
participant "<font color=blue>\nKeyCloak" as idp

autonumber "<b>[00]"
== Запуск приложения ==

  frontend -> frontend: Определить уникальный device id\nсобрать данные env
data (устр-во, система, приложение)

  frontend -> frontend: Принять решения об упрощенной авторизации\n*есть
device-id\n*есть device key\n*есть user-id\n*есть refresh token\n*дата
экспирации refresh token не вышла\n*есть хэш PIN

  opt Не упрощенная автоирзация
    break
      frontend ->[:Регистрация/Авторизация\n(другой процесс)
    end break
  end opt

== Упрощенная авторизация пользователя ==

  frontend -> auth: Старт процесса, POST /auth/init\n*device-id\n*user-
id\n*env data
  group Принять решение о сессии авторизации
    auth -> auth: (Опционально) Оценка риска: \nпроверить версию приложения,
IP, частоту запросов
    auth -> auth: (Опционально) Оценка риска: \nпроверить black lists, данные
по устройству, системе

    auth -> user: Проверить наличие пользователя по user-id
```

```
    user -> auth: Пользователь есть

    auth -> device: Проверить наличие устройства и привязку устройства к
пользователю
    device -> auth: Все ок

    auth -> auth: Проверить правила на упрощенный вход\n*последний вход <n
дней\n*пользвоатель не блокирован\n*устройство не блокировано\n*нет
подозрительной активности и др.

    auth -> auth: Создать сессию, session-id с TTL (10 мин?)\nсохранить
состояние (step, env data, device-id, device public key, user-id, проверки и
др.)

    auth -> frontend: OK\n*session-id\n*session ttl\n*next_step =
IDENTIFICATION_PIN
  end

  frontend -> frontend: Форма ввода PIN (проверка)
  frontend -> frontend: Создать хэш по PIN, сравнить с хэшом на устройстве
  frontend -> frontend: Расшифровать refresh token по PIN (IOS SecureEnclave)
  frontend -> frontend: Подпись device id ключом device private key

  frontend -> auth: Завершить авторизацию (refresh), POST
/auth/refresh\n*session-id\n*device public key\n*signed device id\n*refresh
token
  group Принять решение о выпуске токена
    auth -> auth: Получить данные по session id и проверить состояние

    auth -> auth: Инвалидировать session-id

    auth -> auth: Проверить, что device public key тот же,\nчто и на
устройстве в бд (данные сессии)
    auth -> auth: Проверить подпись signed device id по device public
key\nпроверить device-id (данные сессии)

    auth -> idp: Выпустить токены
    idp -> auth: access token, refresh token

    auth -> frontend: OK\n*access token\n*refresh token\n*device-id\n*user-id

  end

  frontend -> frontend: Запрос ключа шифрования по PIN\n(IOS SecureEnclave)
  frontend -> frontend: Шифрование refresh token
  frontend -> frontend: Сохранить данные в защищенном хранилище\n*refresh
token (encrypted)\n*метаданные шифрования
  frontend -> frontend: Сохранить в памяти\n*access token
  frontend -> frontend: Сохранить метаданные\n*refresh token expiration

@enduml
```


Front
Auth Gateway
Device Binding Service
User Service
KeyCloak
Запуск приложения
[01] Определить уникальный device id
собрать данные env data (устр-во, система, приложение)
Принять решения об упрощенной авторизации
• есть device-id
• есть device key
[02] • есть user-id
• есть refresh token
• дата экспирации refresh token не вышла
• есть хэш PIN
opt [Не упрощенная авторизация]
break
[03] Регистрация/Авторизация (другой процесс)
Упрощенная авторизация пользователя
Старт процесса, POST /auth/init
[04] • device-id
• user-id
• env data
Принять решение о сессии авторизации
[05] (Опционально) Оценка риска:
проверить версию приложения, IP, частоту запросов
[06] (Опционально) Оценка риска:
проверить black lists, данные по устройству, системе
[07] Проверить наличие пользователя по user-id
[08] Пользователь есть
[09] Проверить наличие устройства и привязку устройства к пользователю
[10] Все ок
Проверить правила на упрощенный вход
• последний вход <n дней
[11] • пользователь не блокирован
• устройство не блокировано
• нет подозрительной активности и др.
[12] Создать сессию, session-id с TTL (10 мин?)
сохранить состояние (step, env data, device-id, device public key, user-id, проверки и др.)
OK
[13] • session-id
• session ttl
• next_step = IDENTIFICATION_PIN
[14] Форма ввода PIN (проверка)
[15] Создать хэш по PIN, сравнить с хэшом на устройстве
[16] Расшифровать refresh token по PIN (IOS SecureEnclave)
[17] Подпись device id ключом device private key
Завершить авторизацию (refresh), POST /auth/refresh
• session-id
[18] • device public key
• signed device id
• refresh token
Принять решение о выпуске токена
[19] Получить данные по session id и проверить состояние
[20] Инвалидировать session-id
[21] Проверить, что device public key тот же,
что и на устройстве в бд (данные сессии)
[22] Проверить подпись signed device id по device public key
проверить device-id (данные сессии)
[23] Выпустить токены
[24] access token, refresh token
OK
• access token
[25] • refresh token
• device-id
• user-id
[26] Запрос ключа шифрования по PIN
(IOS SecureEnclave)
[27] Шифрование refresh token
Сохранить данные в защищенном хранилище
[28] • refresh token (encrypted)
• метаданные шифрования
[29] Сохранить в памяти
• access token
[30] Сохранить метаданные
• refresh token expiration
Front
Auth Gateway
Device Binding Service
User Service
KeyCloak

C4 Container [авторизация]

Ссылка: https://drive.google.com/file/d/1A8HQ1v_Ac3XnszRoQc1bqEbhmDVoMTih/view?usp=sharing



Внутренние и внешние сервисы работают с **Auth Gateway**, который оркестрирует процессы регистрации и авторизации. **Auth Gateway** может работать по-разному для first-party и third-party сервисов.

Например, для third party может быть реализован классический OAuth/authorization code с открытием web-страницы и редиректами, тогда как для first party этого не требуется.

Основные сервисы платформы AnorID:

- **API Gateway -** роутинг АПИ, rate limiting, первичная проверка токенов и ограничение прав на АПИ (проверка client_id, iss, acr/amr, scopes)
- **Auth Gateway** - оркестратор регистрации и авторизации
- **User Management Service** - аккаунты и данные пользвоателей в системах, логины, пользовательские настройки
- **Identity Core Service** - golden record, каналы, trust hierarchy и др.
- **Device Binding Service** - данные по устройствам пользователей, степень доверия, black list, связь user - device - app
- **Identity Provider** - учетные записи, выдача и проверка токенов

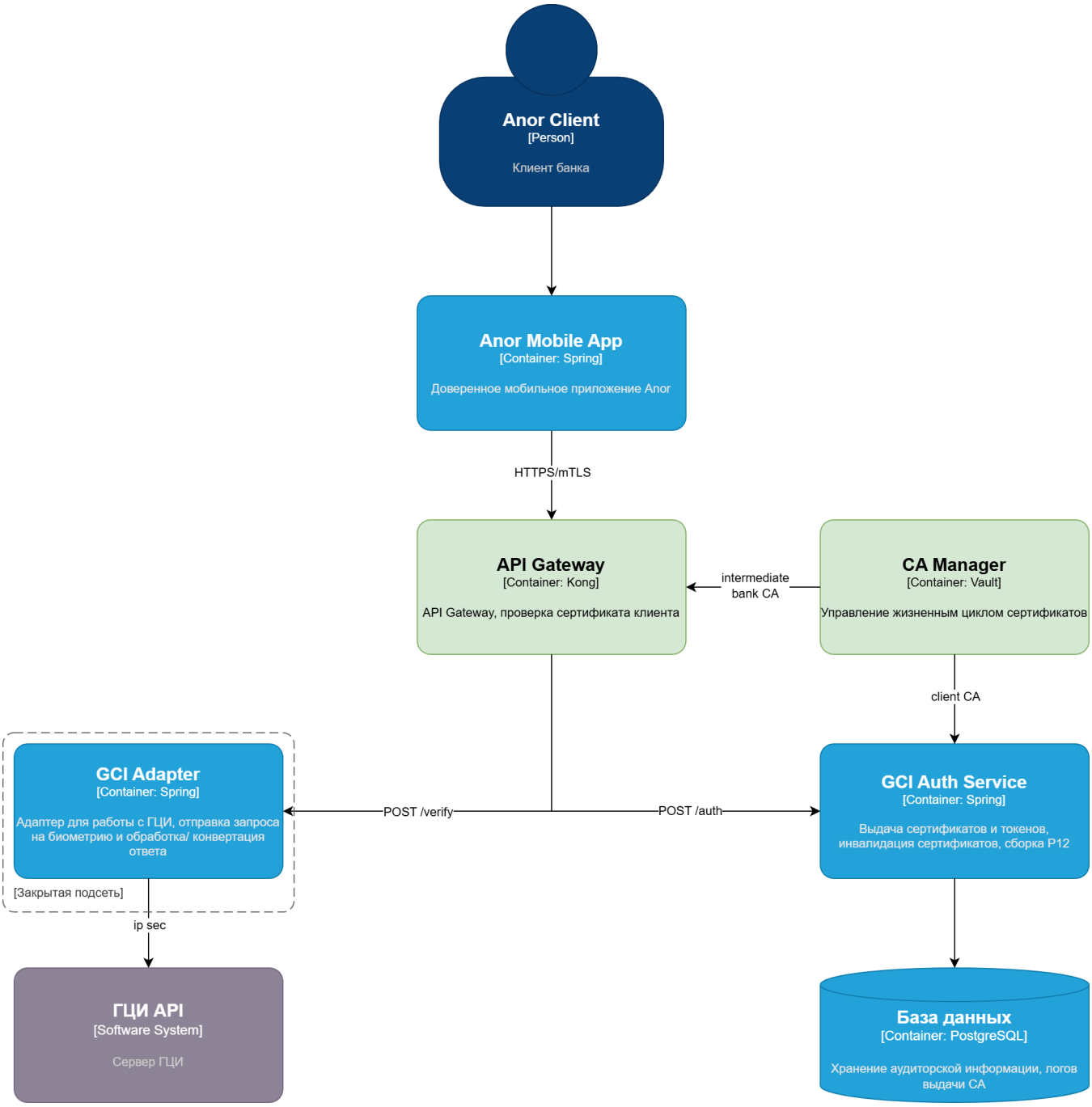
- **Consent Registry Service** - управление согласиями пользователей
- **PDP (Policy Decision Point) Service** - политики доступа, правила авторизации (прав), step-up логика
- **Identity Matching Service** - логика объединения профилей пользователей в один

Допущения:

- **PDP Service** может быть компонентом внутри **Auth Gateway** для упрощения runtime проверок (можно будет вынести позже)

C4 Container [центробанк ГЦИ]

Ссылка: https://drive.google.com/file/d/1531FZ74PkU7-22Q_g7Jsc_1b6UxiSLBq/view?usp=sharing



Компонеты

Платформенные компоненты:

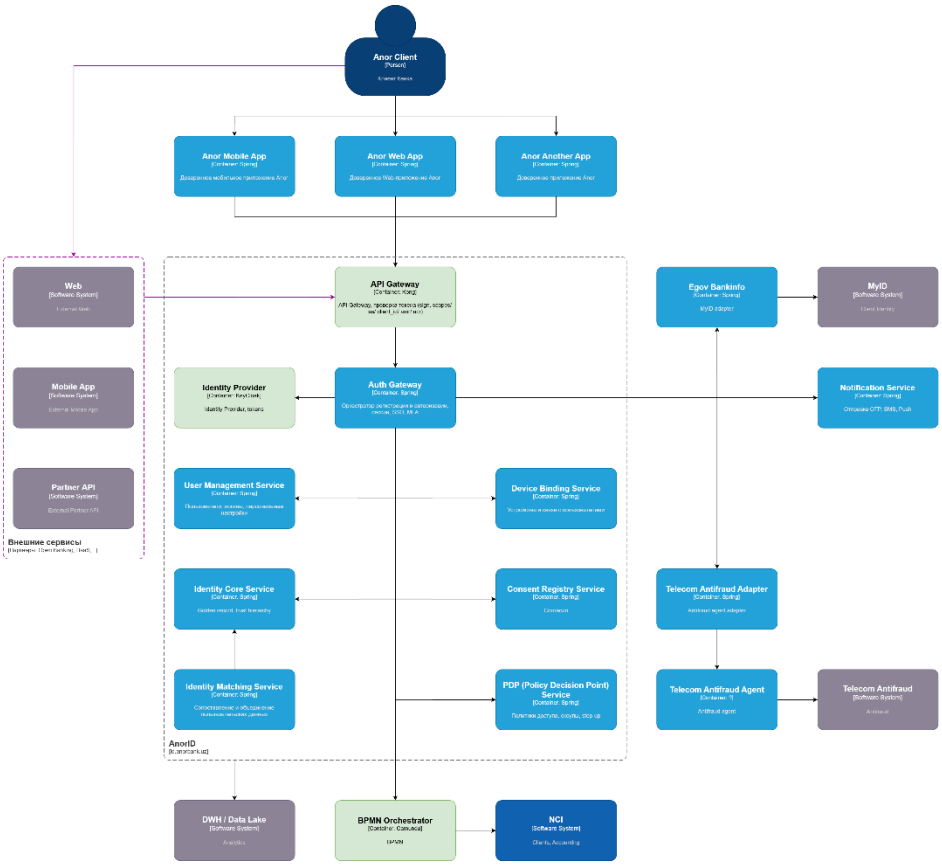
- **Api Gateway (Kong)** - роутинг АПИ, rate limiting, mTLS, проверка клиентских сертификов, (опционально) проверка access token от GCI Auth Service
    - mTLS и проверка сертификатов: плагин mtls-auth
    - нужно импортировать bank intermediate CA (public key) из Vault
- **CA Manager (Vault)** - управление CA, создание клиентских сертификатов, хранение инвалидированных сертификатов
    - нужно создать bank intermediate CA

Продуктовые компоненты (к разработке):

- GCI Adapter- адаптер для работы с АПИ Центробанка (ГЦИ), отправка запросов, обработка ответов, поддержка IP SEC туннеля
- GCI Auth Service - регистрация verification id, выдана access token, выдача клиентского сертификата, упаковка P12 контейнера (для SDK)
- База данных - хранение аудиторских записей по выдаче сертификатов

C4 Container [авторизация]

Ссылка:
https://drive.google.com/file/d/1A8HQ1v_Ac3XnszRoQc1bqEbhmDVoMTih/view?usp=sharing



Внутренние и внешние сервисы работают с **Auth Gateway**, который оркестрирует процессы регистрации и авторизации. **Auth Gateway** может работать по-разному для first-party и third-party сервисов.

Например, для third party может быть реализован классический OAuth/authorization code с открытием web-страницы и редиректами, тогда как для first party этого не требуется.

Основные сервисы платформы AnorID:

- **API Gateway -** роутинг АПИ, rate limiting, первичная проверка токенов и ограничение прав на АПИ (проверка client_id, iss, acr/amr, scopes)

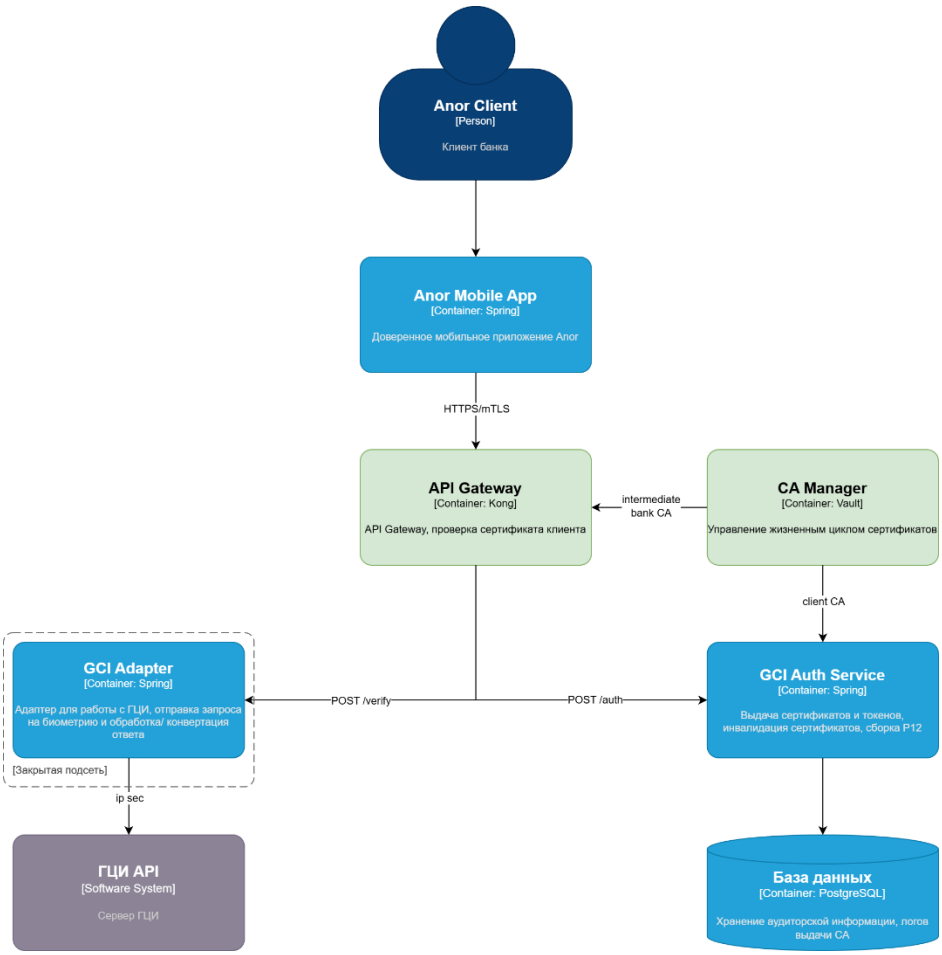
- **Auth Gateway** - оркестратор регистрации и авторизации
- **User Management Service** - аккаунты и данные пользвоателей в системах, логины, пользовательские настройки
- **Identity Core Service** - golden record, каналы, trust hierarchy и др.
- **Device Binding Service** - данные по устройствам пользователей, степень доверия, black list, связь user - device - app
- **Identity Provider** - учетные записи, выдача и проверка токенов
- **Consent Registry Service** - управление согласиями пользователей
- **PDP (Policy Decision Point) Service** - политики доступа, правила авторизации (прав), step-up логика
- **Identity Matching Service** - логика объединения профилей пользователей в один

Допущения:

- **PDP Service** может быть компонентом внутри **Auth Gateway** для упрощения runtime проверок (можно будет вынести позже)

C4 Container [центробанк ГЦИ]

Ссылка: https://drive.google.com/file/d/1531FZ74PkU7-22Q_g7Jsc_1b6UxiSLBq/view?usp=sharing



Компонеты

Платформенные компоненты:

- **Api Gateway (Kong)** - роутинг АПИ, rate limiting, mTLS, проверка клиентских сертификов, (опционально) проверка access token от GCI Auth Service
    - mTLS и проверка сертификатов: плагин mtls-auth
    - нужно импортировать bank intermediate CA (public key) из Vault
- **CA Manager (Vault)** - управление CA, создание клиентских сертификатов, хранение инвалидированных сертификатов
    - нужно создать bank intermediate CA

Продуктовые компоненты (к разработке):

- **GCI Adapter**- адаптер для работы с АПИ Центробанка (ГЦИ), отправка запросов, обработка ответов, поддержка IP SEC туннеля
- **GCI Auth Service** - регистрация verification id, выдана access token, выдача клиентского сертификата, упаковка P12 контейнера (для SDK)
- **База данных** - хранение аудиторских записей по выдаче сертификатов

PIN-код для входа в приложение и его проверка

Рассматриваются разные варианты реализации входа в приложение по PIN-коду с выбором наилучшего.

Термины:

- **PIN** - короткий числовой код, задается пользвоателем
- **Access token** - короткоживущий токен
- **Refresh token** - долгоживущий токен, для выдачи access token
- **Device private key** — не экспортируемый приватный ключ, генерируемый приложением на устройстве (Secure Enclave / Keystore)
- **Device public key** — экспортируемый публичный ключ, генерируемый приложением в связке с device private key

Рекомендация: использование вырианта 2.1 как наиболее безопасного.

Варианты

1. PIN передается на сервер

- Приложение передает PIN на сервер при его генерации, в хэшированном виде.
- Сервер сохраняет PIN.
- При дальнейших вводах PIN сервер сверяет хэши PIN и выдает access token / refresh token.

Плюсы

- простая логика, легко понять и реализовать

Минусы

- сервер знает PIN
- PIN передается по сети и участвует в авторизации
- PIN не привязан к устройству/инстансу приложения
- потенциально повод для угрозы OWASP https://mas.owasp.org/MASWE/MASVS-AUTH/MASWE-0043/ (потенциально из-за beta-версии)

Угрозы по STRIDE

STRIDE	Угроза	Комментарий
Spoofing	Подбор PIN	PIN короткий и только числа
Tampering	Подмена PIN в запросе	
Information Disclosure	Утечка PIN из контура банка	

2. PIN не передается на сервер

2.1 Использование ключей на устройстве, PIN предоставляет доступ до ключей

- Приложение генерирует пару ключей (device private key, device public key) при установке PIN
- Приложение, сохраняет device private key в системное хранилище и задает доступ до операции использования ключа через PIN
- Приложение передает device public key в связке с id устройства (может генерироваться при установке приложения) на сервер
- Сервер сохраняет связь устройства и device public key
- При дальнейших вводах PIN:
    - приложение запрашивает с сервера короткоживующий код авторизации (challenge), с датой экспирации
    - приложение получает доступ до использования device private key через PIN и подписывает им код авторизации
    - приложение отправляет на сервер id устройства, код авторизации и подпись
    - сервер проверяет подпись и ее соответствие устройству (по device public key), срок действия кода, факт использования кода в прошлом (опционально, защита replay)
    - сервер выдает access token / refresh token

Плюсы

- сервер (и банк) ничего не знают про PIN, нет расшаренных секретов
- PIN не участвует в авторизации - только ключи шифрования
- PIN жество привязан к устройству

- проверка PIN и его использование через системное АПИ
- ключи устройства могут быть использованы и в других сценарицях (выдача access по refresh, подпись всех запросов)
- при необходимости данный подход также работает для TouchID/FaceID

Минусы

- сложнее в реализации
- невозможность переиспользовать PIN на другом устройстве (для безопасности плюс, но для пользователя скорее минус)
- PIN не привязан к устройству/инстансу приложения

Угрозы по STRIDE

STRIDE	Угроза
Tampering	Rooted device, поулчние доступа к логике проверки/считывания PIN

2.1 Использование refresh token, PIN предоставляет доступ до refresh token

- Приложение, сохраняет refresh token в системное хранилище и задает доступ до операции его использования через PIN
- При дальнейших вводах PIN:
    - приложение получает доступ до использования refresh token через PIN
    - приложение отправляет на сервер запрос выдачи access token по refresh token
    - сервер выдает access token

Плюсы

- безопасноть лучше чем в варианте 1, реализация проще чем 2.1
- при необходимости данный подход также работает для TouchID/FaceID

Минусы

- refresh token - единственный ключ, кража токена означает получение доступа
- возможен replay
- нет крипто-привязки устройсва как в 2.1
- требуется корректно обрабатывать экспирацию refresh token
- сервер по сути не знает был ли введен PIN (т.к. стандартное АПИ выдачи access token)

Угрозы по STRIDE

STRIDE	Угроза
Spoofing	Использование украденного refresh
Spoofing	Refresh token можно повторно использовать
Tampering	Подмена refresh в памяти

Тем не менее, сами угрозы мягче, чем в п.1

PIN-код для входа в приложение и его проверка

Рассматриваются разные варианты реализации входа в приложение по PIN-коду с выбором наилучшего.

Термины:

- **PIN** - короткий числовой код, задается пользвоателем
- **Access token** - короткоживущий токен
- **Refresh token** - долгоживущий токен, для выдачи access token
- **Device private key** — не экспортируемый приватный ключ, генерируемый приложением на устройстве (Secure Enclave / Keystore)
- **Device public key** — экспортируемый публичный ключ, генерируемый приложением в связке с device private key

Рекомендация: использование вырианта 2.1 как наиболее безопасного.

Варианты

1. PIN передается на сервер

- Приложение передает PIN на сервер при его генерации, в хэшированном виде.
- Сервер сохраняет PIN.
- При дальнейших вводах PIN сервер сверяет хэши PIN и выдает access token / refresh token.

Плюсы

- простая логика, легко понять и реализовать

Минусы

- сервер знает PIN
- PIN передается по сети и участвует в авторизации
- PIN не привязан к устройству/инстансу приложения
- потенциально повод для угрозы OWASP https://mas.owasp.org/MASWE/MASVS-AUTH/MASWE-0043/ (потенциально из-за beta-версии)

Угрозы по STRIDE

STRIDE	Угроза	Комментарий
Spoofing	Подбор PIN	PIN короткий и только числа
Tampering	Подмена PIN в запросе	
Information Disclosure	Утечка PIN из контура банка	

2. PIN не передается на сервер

2.1 Использование ключей на устройстве, PIN предоставляет доступ до ключей

- Приложение генерирует пару ключей (device private key, device public key) при установке PIN
- Приложение, сохраняет device private key в системное хранилище и задает доступ до операции использования ключа через PIN
- Приложение передает device public key в связке с id устройства (может генерироваться при установке приложения) на сервер
- Сервер сохраняет связь устройства и device public key
- При дальнейших вводах PIN:
    - приложение запрашивает с сервера короткоживующий код авторизации (challenge), с датой экспирации
    - приложение получает доступ до использования device private key через PIN и подписывает им код авторизации
    - приложение отправляет на сервер id устройства, код авторизации и подпись
    - сервер проверяет подпись и ее соответствие устройству (по device public key), срок действия кода, факт использования кода в прошлом (опционально, защита replay)
    - сервер выдает access token / refresh token

Плюсы

- сервер (и банк) ничего не знают про PIN, нет расшаренных секретов
- PIN не участвует в авторизации - только ключи шифрования
- PIN жество привязан к устройству
- проверка PIN и его использование через системное АПИ
- ключи устройства могут быть использованы и в других сценарицях (выдача access по refresh, подпись всех запросов)
- при необходимости данный подход также работает для TouchID/FaceID

Минусы

- сложнее в реализации

- невозможность переиспользовать PIN на другом устройстве (для безопасности плюс, но для пользователя скорее минус)
- PIN не привязан к устройству/инстансу приложения

Угрозы по STRIDE

STRIDE	Угроза
Tampering	Rooted device, поулчние доступа к логике проверки/считывания PIN

2.1 Использование refresh token, PIN предоставляет доступ до refresh token

- Приложение, сохраняет refresh token в системное хранилище и задает доступ до операции его использования через PIN
- При дальнейших вводах PIN:
    - приложение получает доступ до использования refresh token через PIN
    - приложение отправляет на сервер запрос выдачи access token по refresh token
    - сервер выдает access token

Плюсы

- безопасноть лучше чем в варианте 1, реализация проще чем 2.1
- при необходимости данный подход также работает для TouchID/FaceID

Минусы

- refresh token - единственный ключ, кража токена означает получение доступа
- возможен replay
- нет крипто-привязки устройсва как в 2.1
- требуется корректно обрабатывать экспирацию refresh token
- сервер по сути не знает был ли введен PIN (т.к. стандартное АПИ выдачи access token)

Угрозы по STRIDE

STRIDE	Угроза
Spoofing	Использование украденного refresh
Spoofing	Refresh token можно повторно использовать
Tampering	Подмена refresh в памяти

Тем не менее, сами угрозы мягче, чем в п.1